



Deusto

Facultad de Ingeniería
Ingeniaritza Fakultatea



Fundamentos del Procesamiento de Lenguaje Natural

Técnicas Avanzadas de Machine Learning para el Procesamiento del Lenguaje Natural



Índice

- Shallow learning approaches.
- Multilayer perceptron.
- Convolutional Neural Networks (CNNs)
- Recurrent neural networks, GRU, LSTM.
- Sequence-to-Sequence models



Aprendizaje supervisado para NLP

Hemos visto cómo generar features que representan el texto de diferentes formas, pero...

¿Cómo podemos usar esas features?



Aprendizaje supervisado para NLP (II)

- Podemos representar el texto:
 - One hot encodings.
 - Bag of Words.
 - TF-IDF.
 - Non Contextual Word Embeddings.
 - Contextual Word embeddings.



Aprendizaje supervisado para NLP (III)

- Y los podemos usar con:
 - Shallow/Traditional Machine Learning → e.g., los modelos disponibles en Scikit-Learn
 - Redes Neuronales Densas (i.e., Multilayer Perceptrons, MLPs)
 - Convolutional Neural Networks (CNNs)
 - Long Short-Term Memory (LSTM) / Bi-LSTM
 - entre otras muchas más opciones...



Text Classification

- Sentiment analysis.
- Asignación de categorías:
 - Spam/ham
 - Fake/Not Fake
 - Detección de temas (Política, Medicina, Marketing...)
 - Polarizado/No polarizado.
- Detección de autoría.
- Identificar diferentes formas de hablar/escribir.
- ...



Text Classification – Definición

- Input:
 - un documento d .
 - una conjunto definido de clases objetivo $C=\{c_1, c_2, \dots, c_j\}$
- Output: la clase que se predice $c \in C$



Supervised text classification

- Input:
 - un documento d .
 - una conjunto definido de clases objetivo $C=\{c_1, c_2, \dots, c_j\}$
 - Un conjunto de entrenamiento de m documentos anotados: $(d_1, c_1), \dots, (d_m, c_m)$
- Output:
 - un clasificador entrenado $\lambda:d \rightarrow c$



Supervised text classification (II)

- **Binary classification**
 - Cada elemento de entrada se asigna exactamente a **una de las 2 clases**.
- **Multi-class classification**
 - Cada elemento de entrada se asigna exactamente a **una de las K clases** ($K > 2$).
- **Multi-label classification**
 - Cada elemento de entrada se asigna a **N de las K clases** ($N \geq 1$, varía según el elemento de entrada).



Shallow learning approaches

- ¿Qué features podemos usar?
 - One hot encodings.
 - Bag-of-Words.
 - TF-IDF.
 - Non Contextual Word Embeddings.
 - Contextual Word embeddings.
- ¿Qué clasificadores podemos usar?
 - Naïve Bayes.
 - Logistic Regression.
 - Decision Trees.
 - Random Forests.
 - SVM
 - XGBoost
 - ...

[Colab.](#)



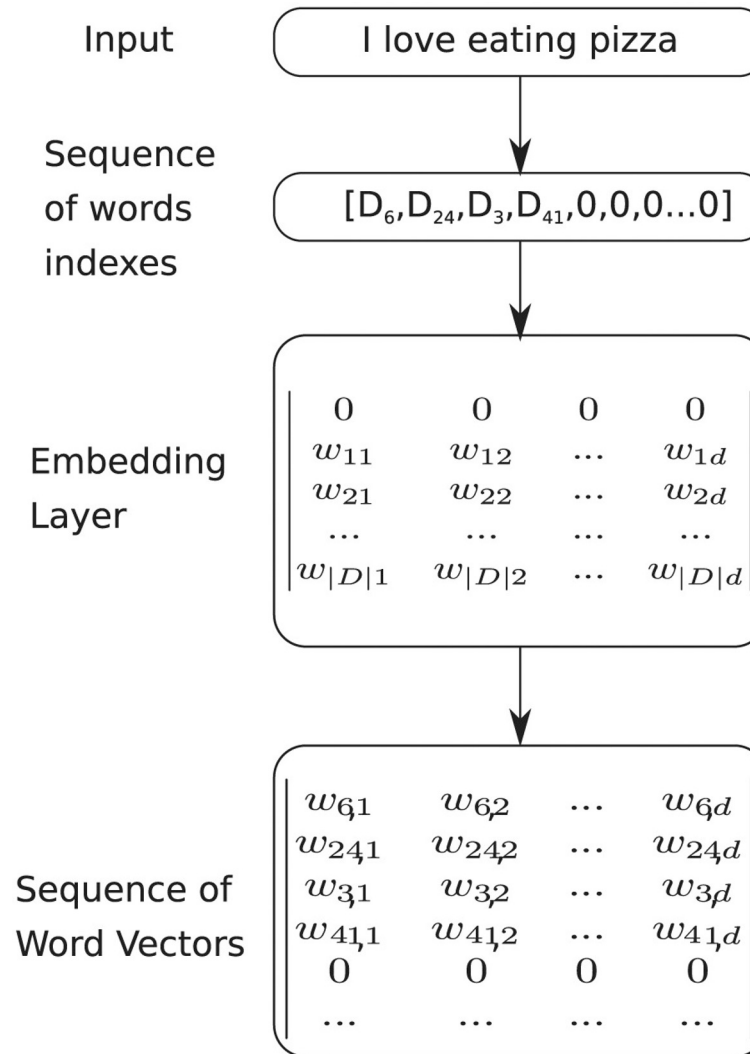
Redes Neuronales

- ¿Qué features podemos usar?
 - One hot encodings.
 - Bag of Words.
 - TF-IDF.
 - Non Contextual Word Embeddings.
 - Contextual Word embeddings.

¿Cómo tenemos que insertar los ejemplos de entrenamiento? ¿En qué formato debería estar el texto?



Redes Neuronales (II)





Redes Neuronales (III)

- **Vocabulary:** lista de palabras que se van a usar en el modelo con su correspondiente ID numérico.
 - Coordina el mapeo **integer-to-token** y viceversa.
 - También se puede usar para mapear los class **labels a integers**.
 - Los one-hot-encodings se pueden generar en base a estos ID numéricos.



Redes Neuronales (IV)

- **Vectorizer.**
 - Encapsula el **Vocabulary** y es el responsable de convertir strings en vectores numéricos. **Palabra** → **Vector**.

¿Qué longitud tiene que tener ese vector?
¿Qué forma para representar las palabras se va a usar?



Redes Neuronales (IV)

- **Vectorizer.**
 - Encapsula el **Vocabulary** y es el responsable de convertir strings en vectores numéricos. **Palabra** → **Vector**.
 - One hot encoding.
 - Word2Vec, Glove, FastText...
 - ¿Contextuales?



YELP Dataset

- Creado para un reto en 2015 → Predecir la nota que los clientes daban a los restaurantes basándose en su review.
 - 1-2 estrellas → Negativo.
 - 3-4 estrellas → Positivo.
- 560,000 ejemplos de entrenamiento y 38,000 ejemplos para test.
 - <https://www.kaggle.com/datasets/yelp-dataset/yelp-dataset>
 - https://www.kaggle.com/datasets/yelp-dataset/yelp-dataset/data?select=yelp_academic_dataset_review.json
- Vamos a trabajar con un subset de todo el corpus.



YELP + Perceptron

- [Colab.](#)



Dividiendo el dataset

¿Qué formas conocéis de dividir un dataset para su entrenamiento y evaluación?



Holdout Method

- El dataset se divide de **manera aleatoria** en dos conjuntos independientes, el **training set** y el **test set**.
 - e.g., **dos tercios** de los datos son usados **training set**, y el resto (**un tercio**) es usado para el **test set**.
 - El **training set** se usa para **entrenar** el modelo.
 - El **accuracy (o cualquier otra métrica)** es calculada con el **test set**.



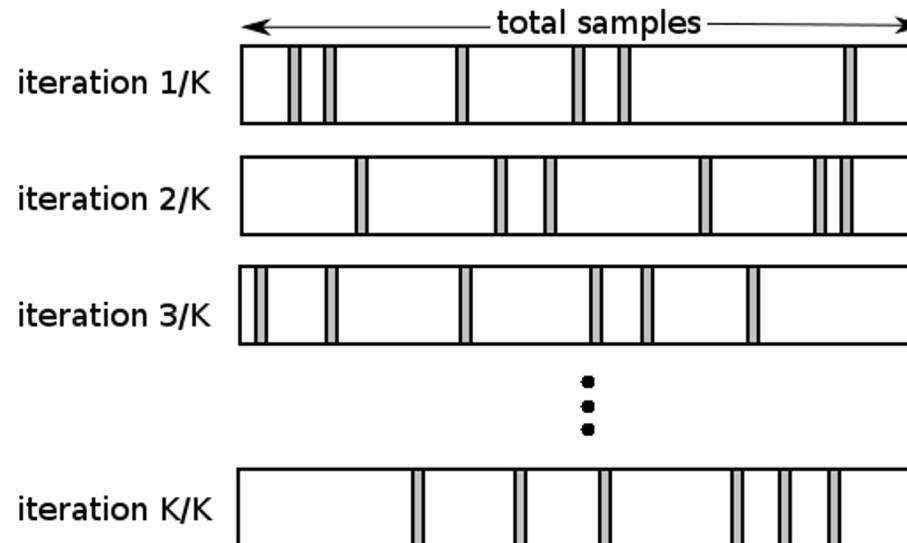
Holdout Method (II)

- El más directo o facil de usar.
 - Usual 70%-30%.
 - [sklearn.model_selection.train_test_split](#)
 - train/test size.
 - random state (la división puede variar de un ordenador a otro dependiendo de versiones de librerías, arquitectura, etc.)
 - **Siempre randomizar.**
- Limitaciones:
 - No hyperparameter tuning.
 - Las decisiones que tomamos a la hora de crear el modelo están sesgadas hacia el test set.
 - ¿Cómo podemos evitar esto?
 - **Ningún proyecto con este enfoque.**



Random Subsampling

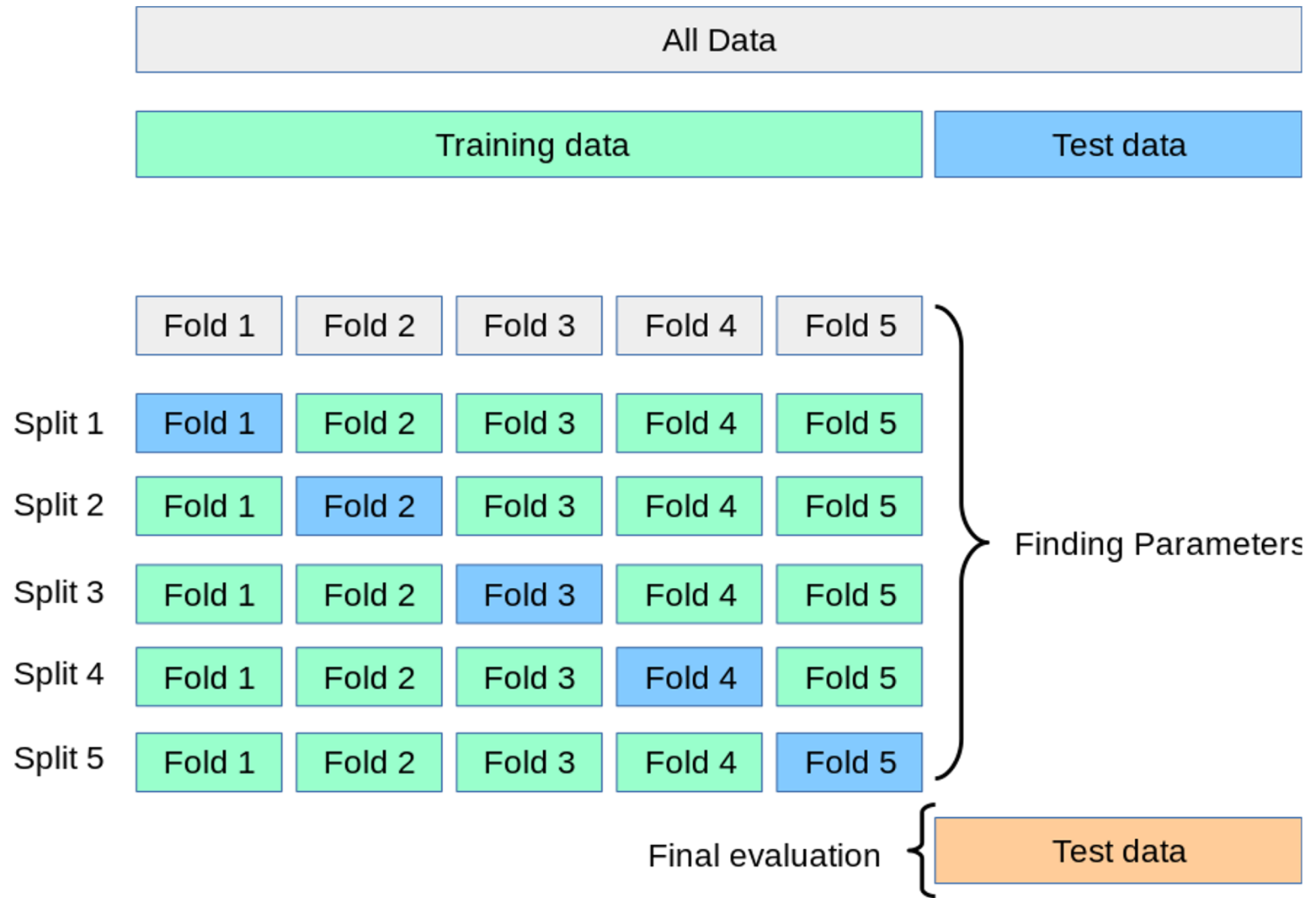
- **Variante** del **holdout** method en el que el **holdout** method es **repetido** k veces.
 - El accuracy general es estimado calculando el promedio del accuracy de cada iteración.



[Source.](#)



Cross Validation



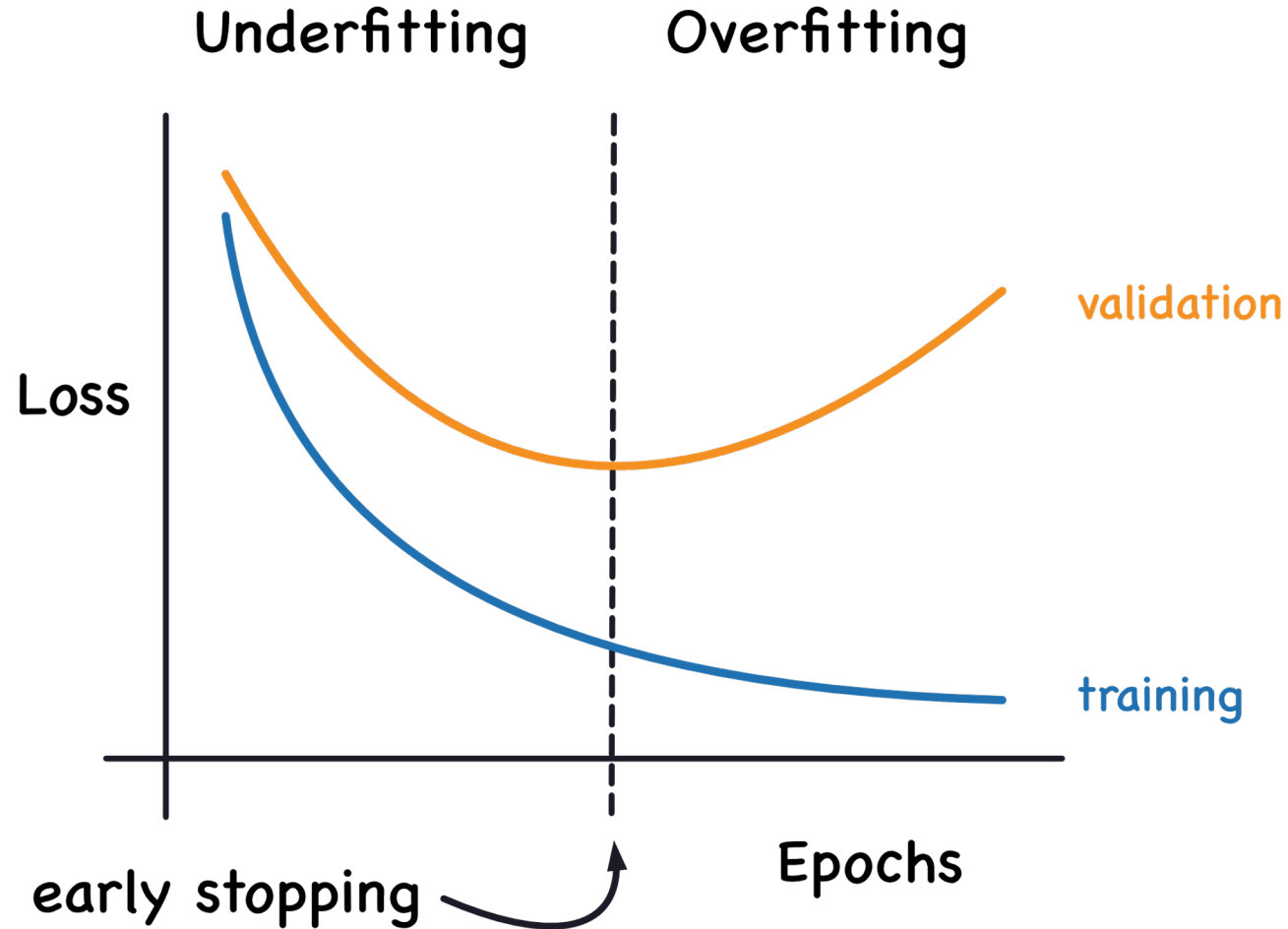


Train-Eval-Test

- Dividimos el conjunto de datos en 3 subconjuntos:
 - Train (70%)
 - Eval (15%) → Cálculo de hyper-parámetros.
 - Test (15%)
- Intentamos mantener la misma **proporción de clases** → Stratification.
- **Early stopping** → Con el validation test...



Early Stopping

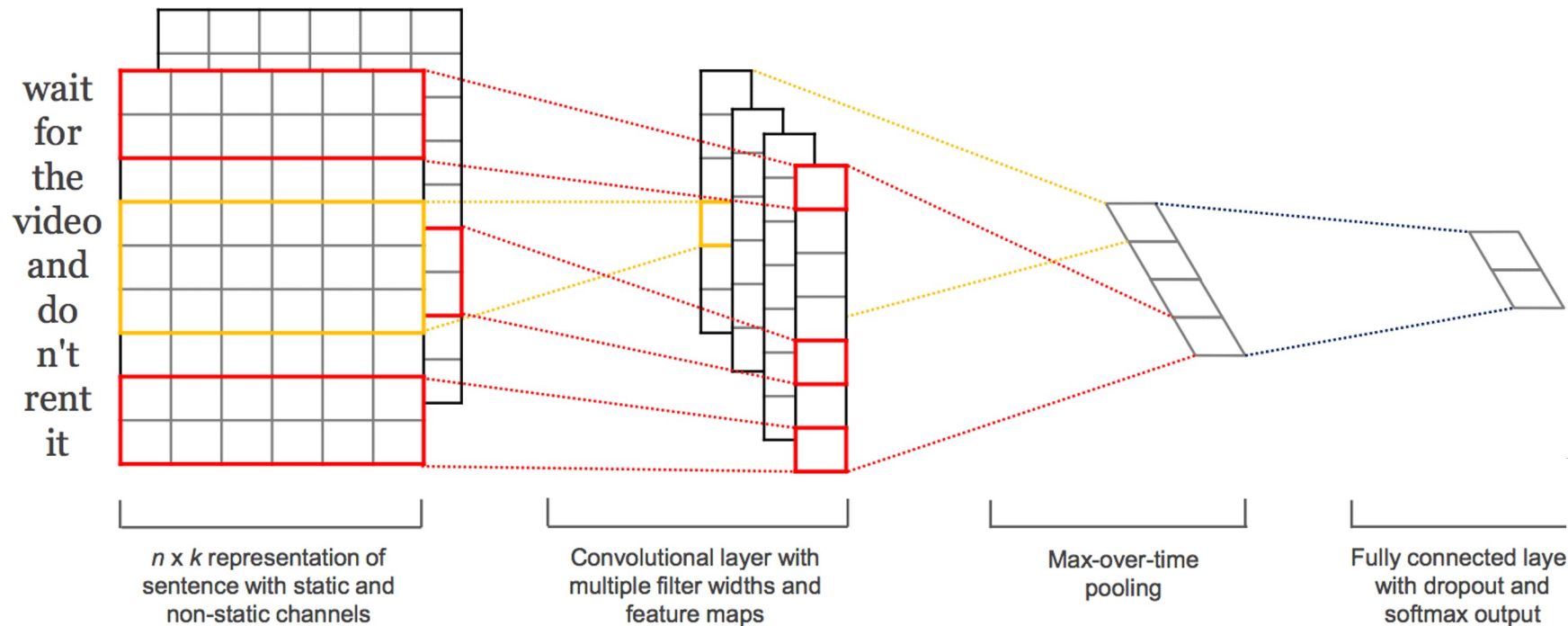


[Source.](#)



Convolutional Neural Networks for Text Classification

- [Convolutional Neural Networks for Sentence classification, Yoon Kim, 2014.](#)



[Source](#)



Convolutional Neural Networks for Text Classification (II)

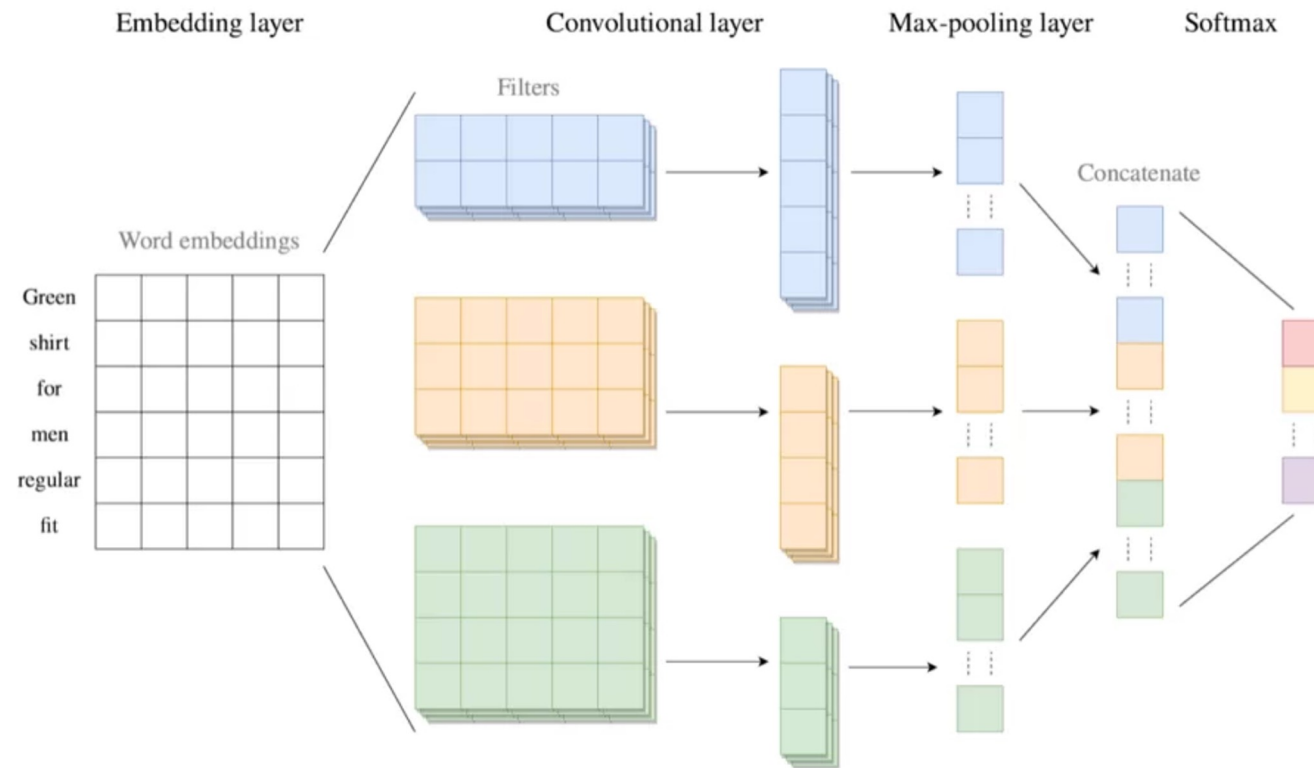


Figure 3: General CNN architecture for text classification

[Source](#)



Convolutional Neural Networks for Text Classification (III)

SequenceVocabulary

```
{  
'<MASK>': 0,  
'<UNK>': 1,  
'<BEGIN-OF-SEQUENCE>': 2,  
'<END-OF-SEQUENCE>': 3,  
'is': 4,  
'happy': 5  
}
```

"Jerry is happy"

1 4 5

2 1 4 5 3

2 1 4 5 3 0 0

Step 0: Input Sentence

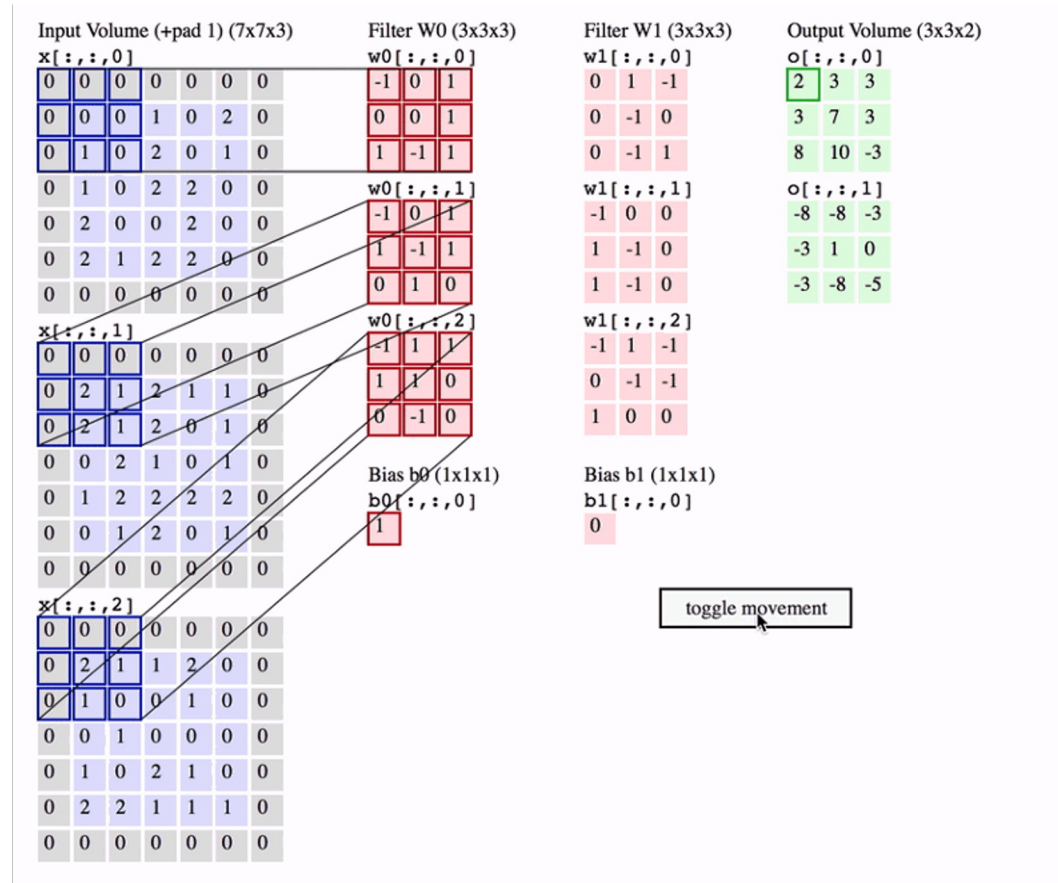
Step 1: Using the Vocabulary to map words to integers

Step 2: Wrapping the sentence with boundary markers

Step 3: Padding with 0s to keep all vectors the same size.



Convolutional Neural Networks for Text Classification (IV)



[Source](#)



Convolutional Neural Networks for Text Classification (V)



[Source](#)



Convolutional Neural Networks for Text Classification (VI)

- [Colab](#) con todo el pipeline de CNNs for Text Classification explicado.



Loss function - Categorical cross-entropy

```
>>> y_true = [[0, 1, 0], [0, 0, 1]]
>>> y_pred = [[0.05, 0.95, 0], [0.1, 0.8, 0.1]]
>>> # Using 'auto'/'sum_over_batch_size' reduction type.
>>> cce = keras.losses.CategoricalCrossentropy()
>>> cce(y_true, y_pred)
1.177
```

$$\text{logloss} = -\frac{1}{N} \sum_i^N \sum_j^M y_{ij} \log(p_{ij})$$

- N is the number of rows
- M is the number of classes

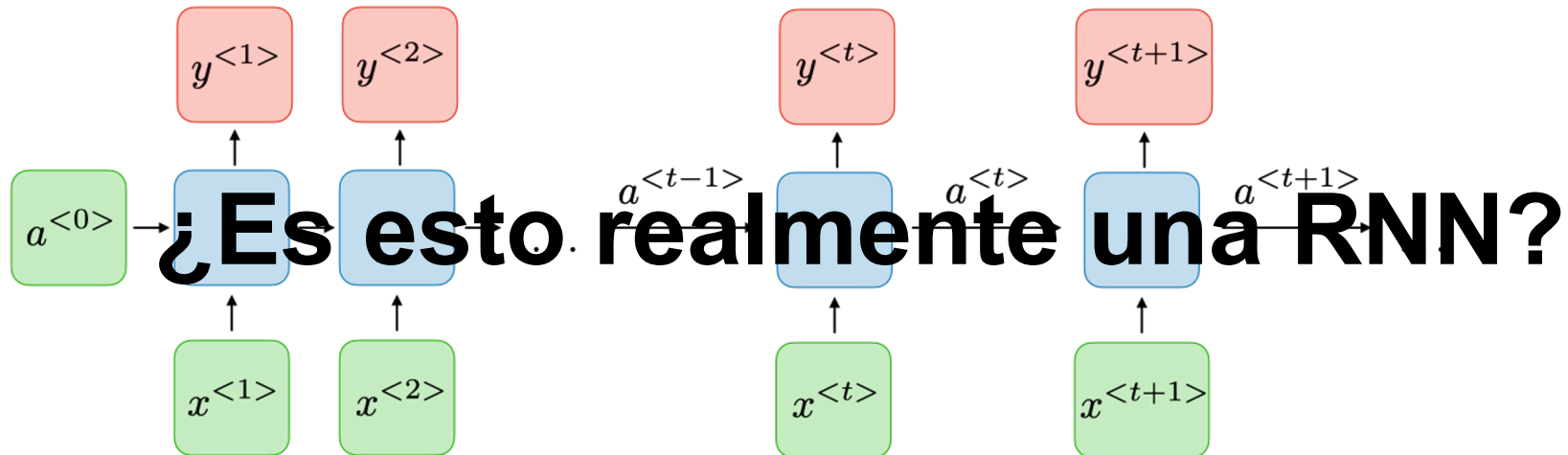


Sequence Modeling for NLP

¿Cuál es el modelo recurrent más básico para modelar secuencias?



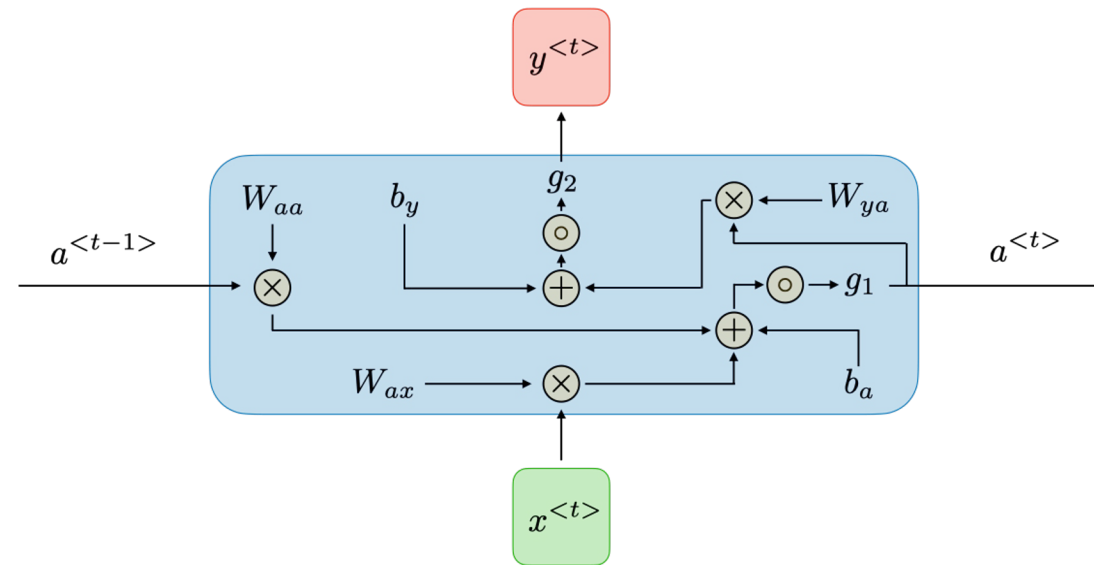
Recurrent Neural Networks (RNNs)



[Source](#)



Recurrent Neural Networks (II)



$$a^{<t>} = g_1(W_{aa}a^{<t-1>} + W_{ax}x^{<t>} + b_a)$$

and

$$y^{<t>} = g_2(W_{ya}a^{<t>} + b_y)$$

[Source](#)



Recurrent Neural Networks (III)

- **Limitaciones:**
 - Incapacidad de retener información para predicciones a largo plazo.
 - La estabilidad del gradiente.
 - **Vanishing Gradient Problem.**

¿Qué inventaron para arreglar estos problemas?



Gating

- Imagina que estás sumando dos cantidades, a y b , pero quieres controlar cuánto de b se incorpora en la suma:
 - $a + b \rightarrow a + \lambda b$
 - λ es un valor entre 0 y 1.
 - Si $\lambda=0$ no hay contribución de b en la suma, si $\lambda=1$ contribuye completamente.
- Podemos interpretar que λ actúa como un “**interruptor**” o “**compuerta**” que **controla la cantidad de b que se incorpora a la suma.**



Gating (II)

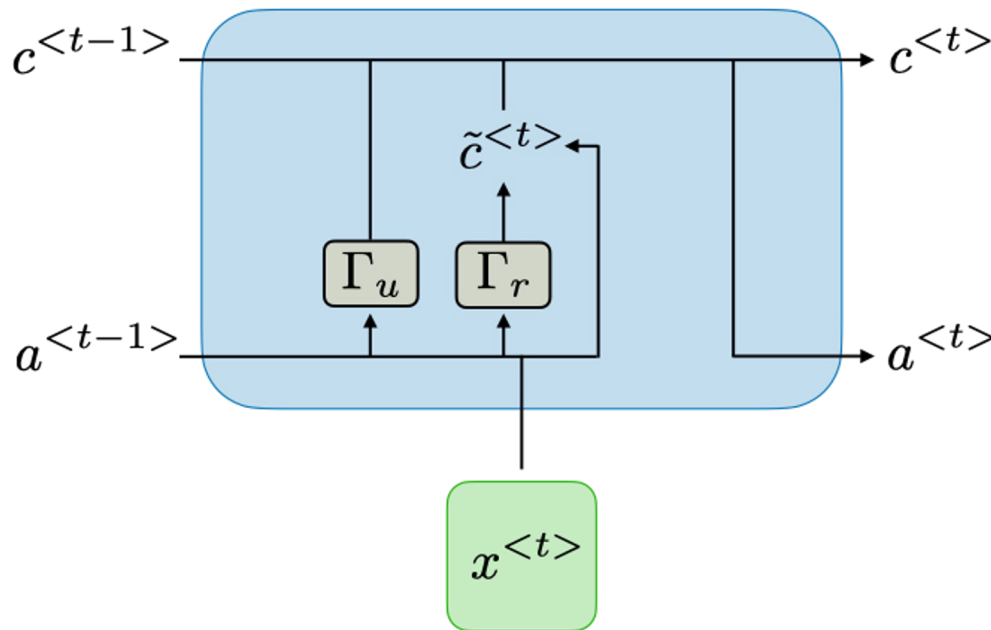
Tipo de Gate	Rol	Usado en
Update gate Γ_u	¿Cuánto debería importar el pasado?	GRU, LSTM
Relevance gate Γ_r	¿Descartar la información previa?	GRU, LSTM
Forget gate Γ_f	¿Borrar una celda o no?	LSTM
Output gate Γ_o	¿Cuánto revelar de una celda?	LSTM

$$\Gamma = \sigma(Wx^{<t>} + Ua^{<t-1>} + b)$$



Gating - GRU y LSTM

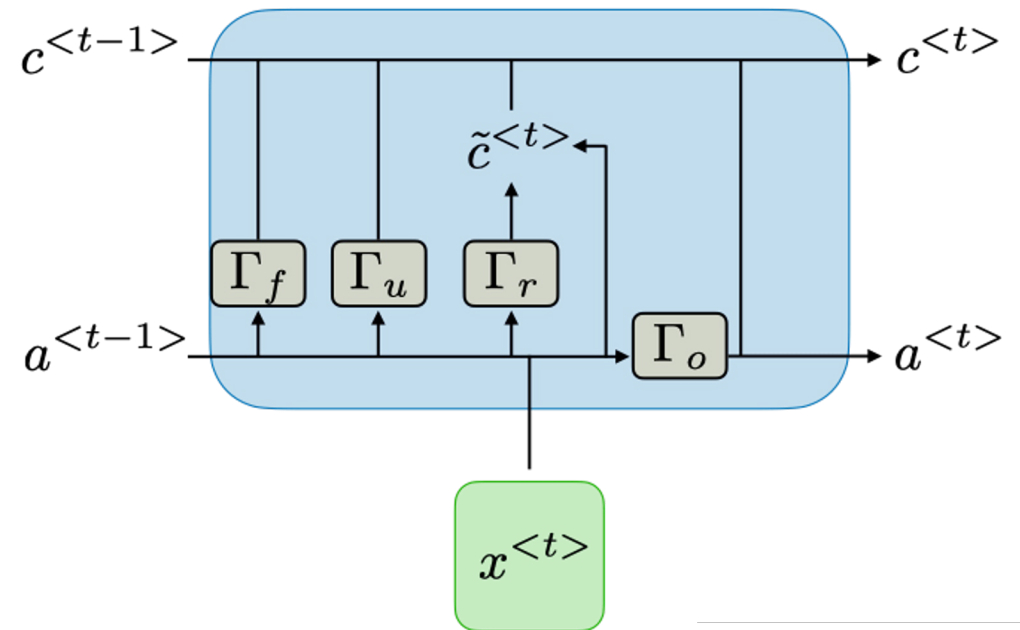
- La **LSTM** puede ser considerada una **generalización** de la **GRU**.



GRU

Explicación matemática disponible en:

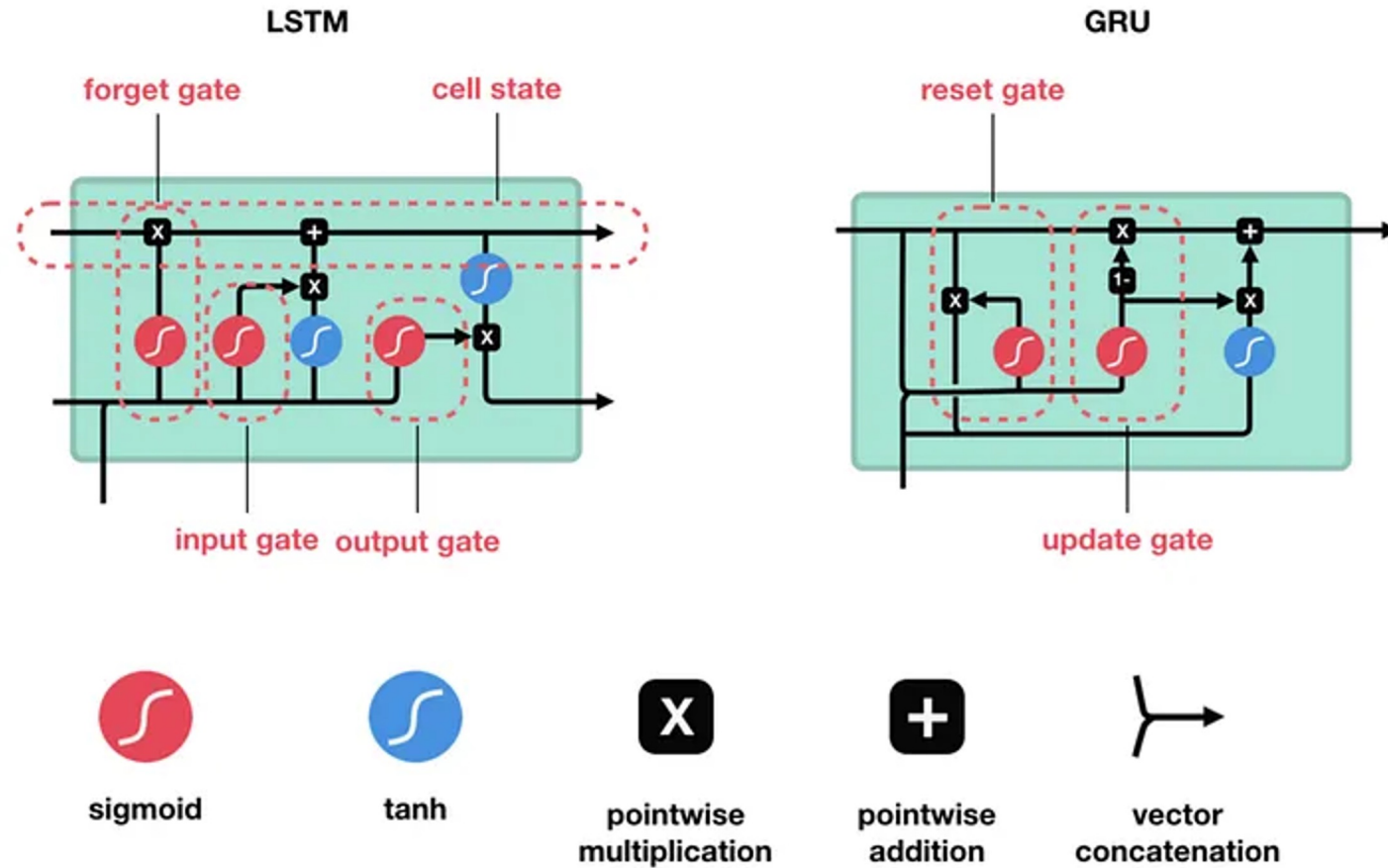
[Source](#)



LSTM



Gating - GRU y LSTM (II)





GRU vs LSTM

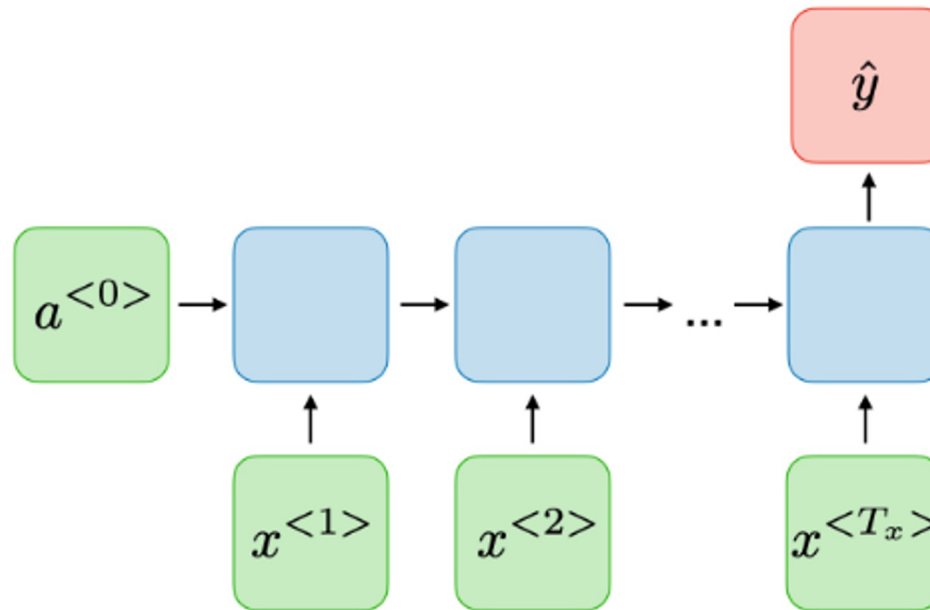
- **GRU** es más **simple comparada** con la **LSTM**: menor número de puertas, menor número de operaciones con tensores, menor número de pesos...
 - **Rule of thumb**: suele funcionar mejor que las LSTM con menor cantidad de datos. A medida que el número de datos se incrementa, las LSTM deberían funcionar mejor.
- **GRU** tiene **2** puertas, LSTM **3** puertas.
- La **diferencia** más **relevante** es la **Output gate** que **controla** la **cantidad** de contenido de **memoria** que se ve o es **utilizada** por **otras unidades** en la red.



Sequence modeling for...

- Sentiment classification

Many-to-one
 $T_x > 1, T_y = 1$

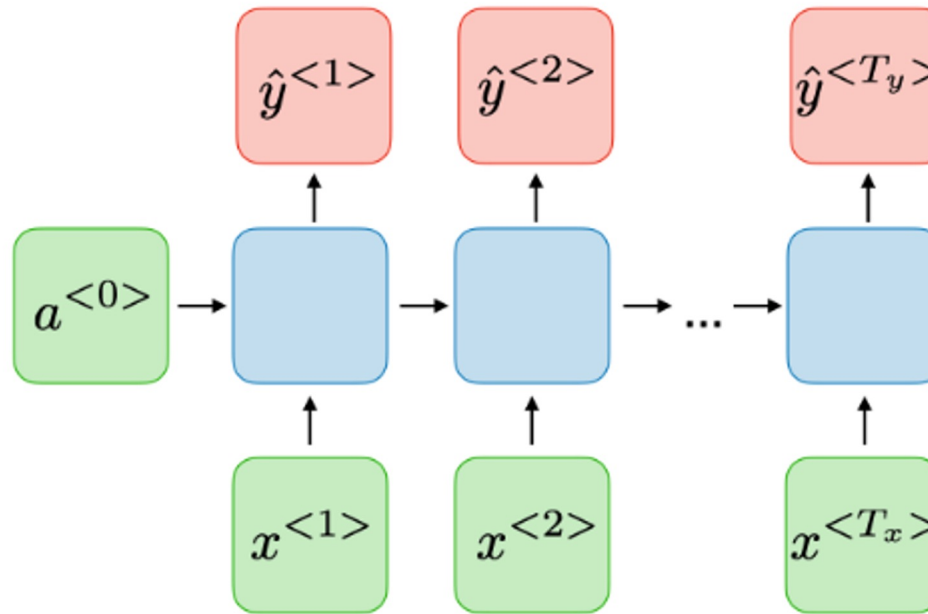




Sequence modeling for...

- Name entity recognition

Many-to-many
 $T_x = T_y$

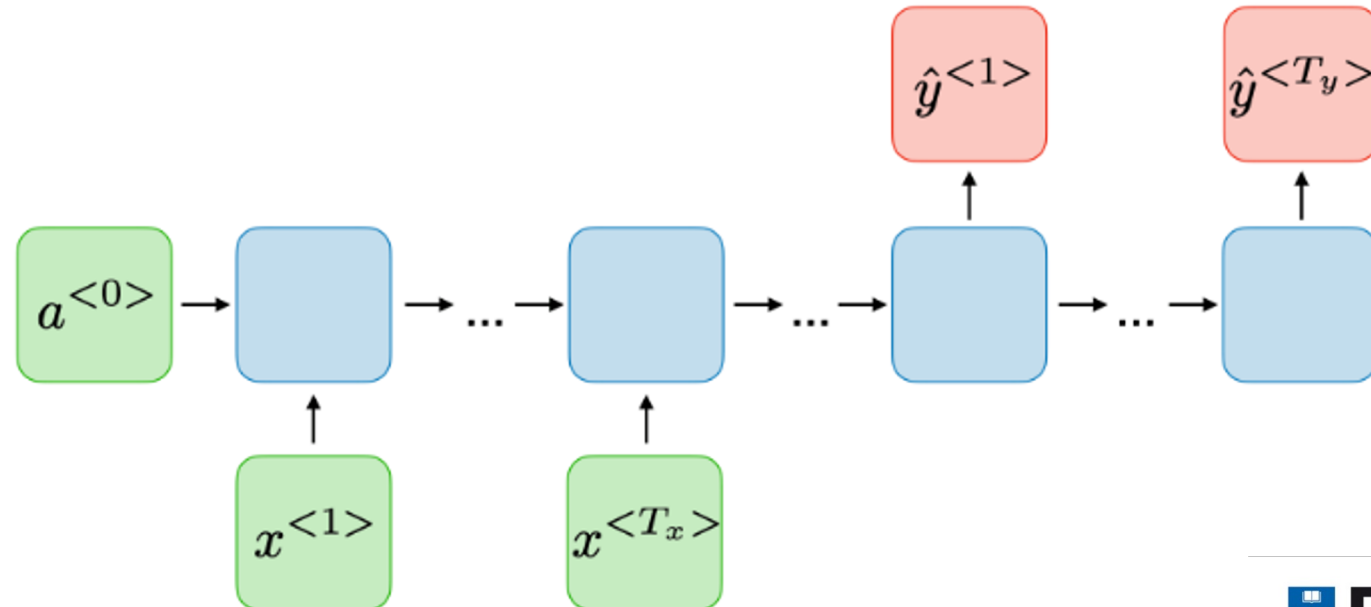




Sequence modeling for...

- Machine translation

Many-to-many
 $T_x \neq T_y$





Sequence modeling for...

- **Generación de apellidos**
 - Dataset con una lista de apellidos y su país de origen.
 - Posibles tareas:
 - Teniendo un apellido predecir su país.
 - Generar apellidos.
 - ...

Hasta ahora hemos trabajado a nivel de palabra...



Sequence modeling for...

...caracteres.



Generación de apellidos usando GRU

- **Dataset**
 - Lista de apellidos con su nacionalidad correspondiente.

surname	nationality
Woodford	English
Coté	French
Kore	English
Koury	Arabic
Lebzak	Russian
Obinata	Japanese
Rahal	Arabic
Zhuan	Chinese
Acconci	Italian
Mifsud	Arabic
Chekhoev	Russian
Essop	English
Rosenbloom	Russian
Michalovicova	Czech



Generación de apellidos usando GRU

- **Transformar los apellidos a secuencias de números.**
 - a. Generación del vocabulario necesario.
 - i. **Tokens reservados:** <MASK>, <UNK>, <BEGIN-OF-SEQUENCE>, <END-OF-SEQUENCE>.
 - ii. **Tokens extraídos del dataset:** todos los caracteres únicos extraídos del train set.
 - b. Creación de un diccionario/mapa caracter $\leftrightarrow$ id numérico.
 - c. Calcular longitud máxima.
 - i. Se requiere para el **padding**.

```
"<MASK>": 0,  
"<UNK>": 1,  
"<BEGIN-OF-SEQUENCE>": 2,  
"<END-OF-SEQUENCE>": 3,  
"A": 4,  
"B": 5,  
"C": 6,  
"D": 7,  
"E": 8,  
"F": 9,  
"G": 10,  
"H": 11,  
"I": 12,  
"J": 13,  
"K": 14,  
"L": 15,  
"M": 16,  
"N": 17,  
"O": 18,  
"P": 19,  
"Q": 20,  
"R": 21,  
"S": 22,  
"T": 23,
```



Generación de apellidos usando GRU

- Transformar los apellidos a secuencias de números.
 - Longitud máxima: 8.
 - Ejemplo: “**Essop**”
 - <BoS>Essop<EoS> → [2, 23, 42, 42, 50, 40, 3, 0]
 - Ejemplo: “**Tano**”
 - <BoS>Tano<EoS> → [2, 30, 55, 66, 50, 3, 0, 0]

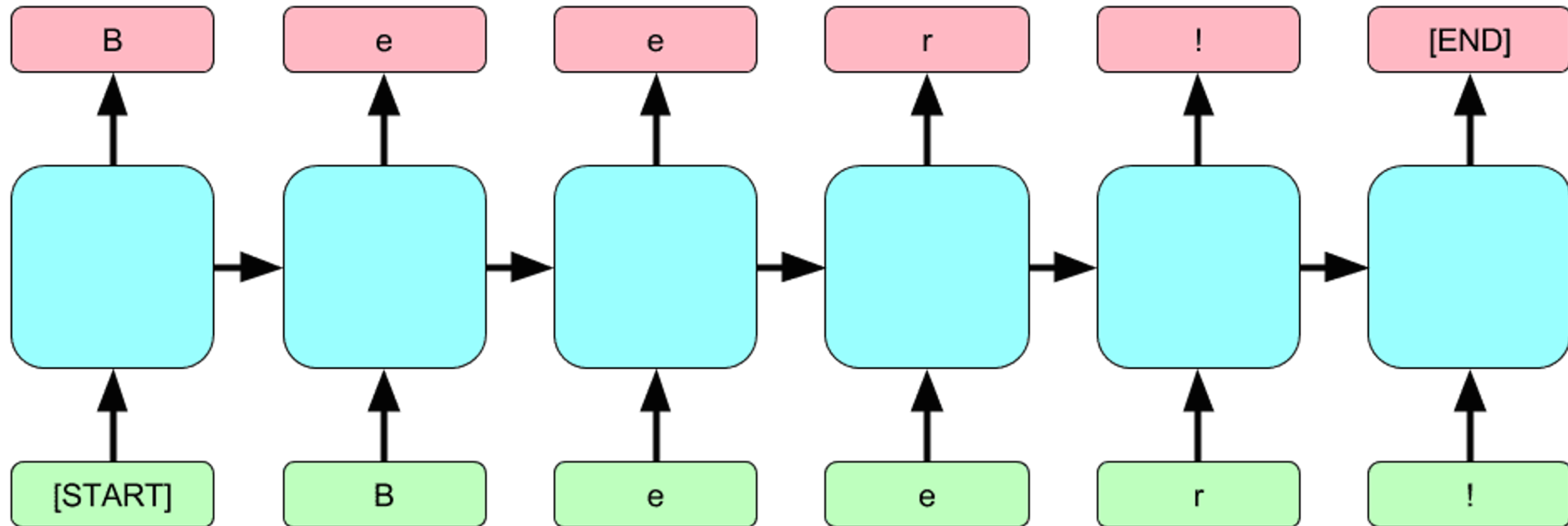


Generación de apellidos usando GRU

¿Cuáles son nuestros ejemplos de entrenamiento? ¿Qué es la X? ¿Qué es la Y?



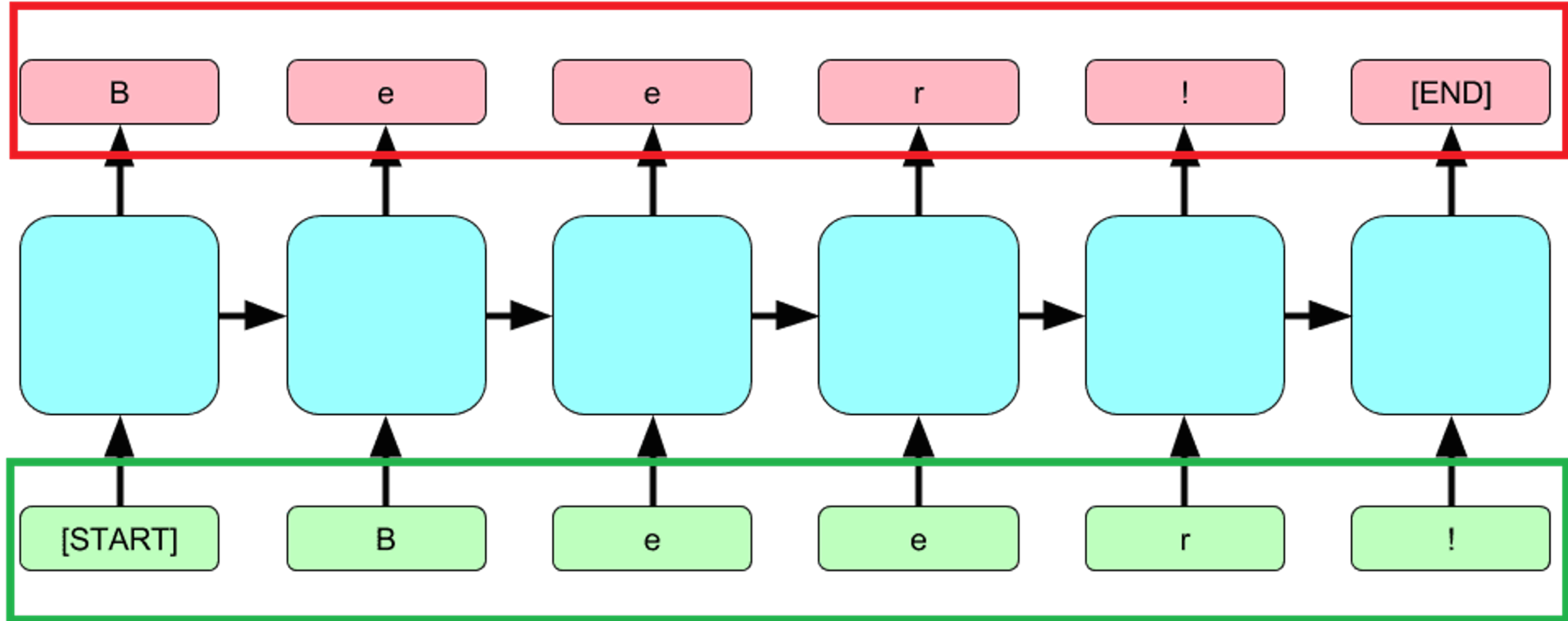
Generación de apellidos usando GRU



[Source](#)



Generación de apellidos usando GRU



[Source](#)

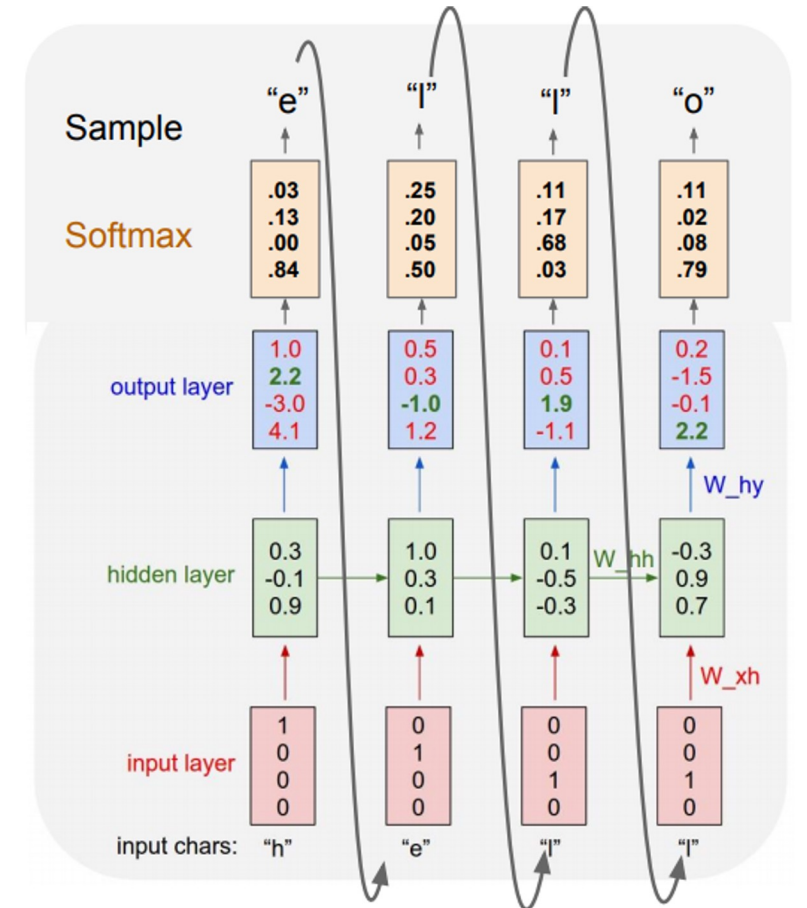


Generación de apellidos usando GRU

- **Transformar los apellidos a ejemplos de entrenamiento: X e Y.**
 - **X:** quitamos el token de final de secuencia y padeamos hasta longitud máxima.
 - **Y:** quitamos el token de inicio de secuencia y padeamos hasta longitud máxima.



Generación de apellidos usando GRU



[Source](#)



Generación de apellidos usando GRU

- **Evaluación de los apellidos generados**

- Se compara posición a posición / caracter a caracter si el caracter predicho es el caracter esperado (**Y_true**).
- Las posiciones de masking son descartados a la hora de calcular el accuracy. Es decir, se divide entre el total de caracteres que no son de masking.

- **Ejemplo:**

- Apellido verdadero (**Y_true**): "Garcia<END-OF-SEQUENCE>" → [10, 32, 17, 32, 38, 32, 3]
- Apellido predicho (**Y_pred**): "Garzia<END-OF-SEQUENCE>" → [10, 32, 17, 53, 38, 32, 3]
- Accuracy = (Número de aciertos) / (Número total de caracteres no masking) = 6/7 = **85.7%**.

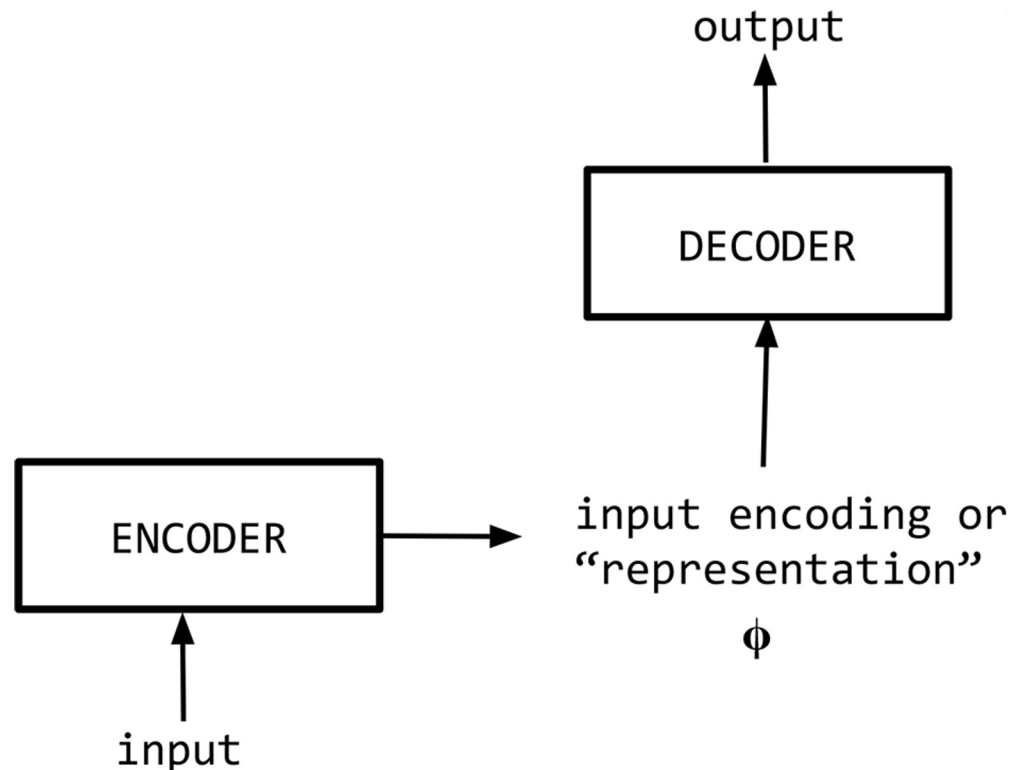
- **¿Y si tenemos en cuenta el masking?**

- Apellido verdadero (**Y_true**): "Garcia<END-OF-SEQUENCE>" → [10, 32, 17, 32, 38, 32, 3, 0, 0, 0]
- Apellido predicho (**Y_pred**): "Garzia<END-OF-SEQUENCE>" → [10, 32, 17, 53, 38, 32, 3, 0, 0, 0]
- Accuracy = (Número de aciertos) / (Número total de caracteres) = 9/10 = **90%**.



Sequence to Sequence Models

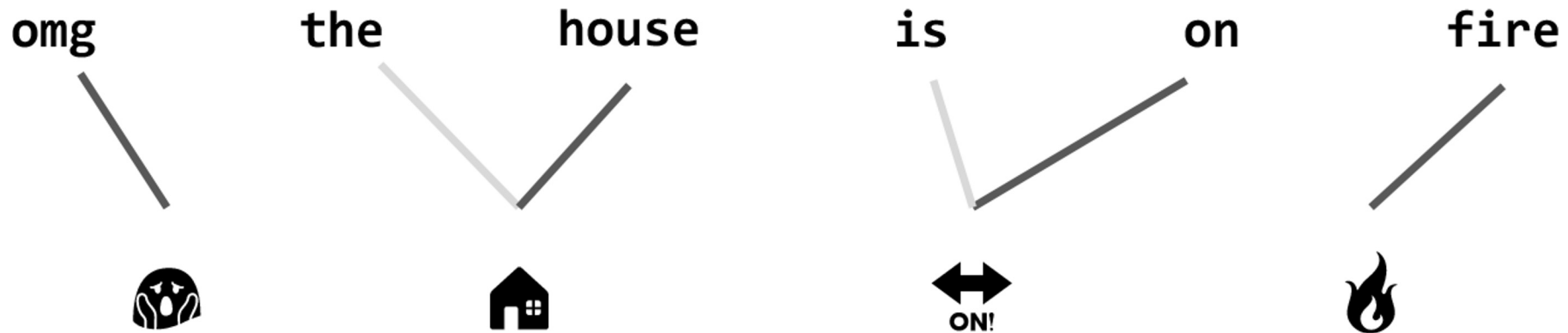
- El modelo tiene una **secuencia** como **input** y **produce** otra **secuencia** como **output** → Posible de una **longitud diferente a la inicial**.
 - Question-answering.
 - Machine Translation.
 - Summarization.





Sequence to Sequence Models (II)

- **Machine translation:** traducción de emojis como problema de predicción de S2S.





Sequence to Sequence Models (III)

- **Machine translation:** traducción de emojis como problema de predicción de S2S.
 - Podemos usar **cualquier** modelo **secuencial** (RNN, GRU o LSTM) para construir el **encoder**.
 - Se suelen usar modelos **bi-direccionales** que combinan **pasadas** hacia **adelante** y hacia **atrás** sobre una secuencia para crear **representaciones** más **ricas**.
 - Por último, se combinan con **mecanismos** de **atención**.

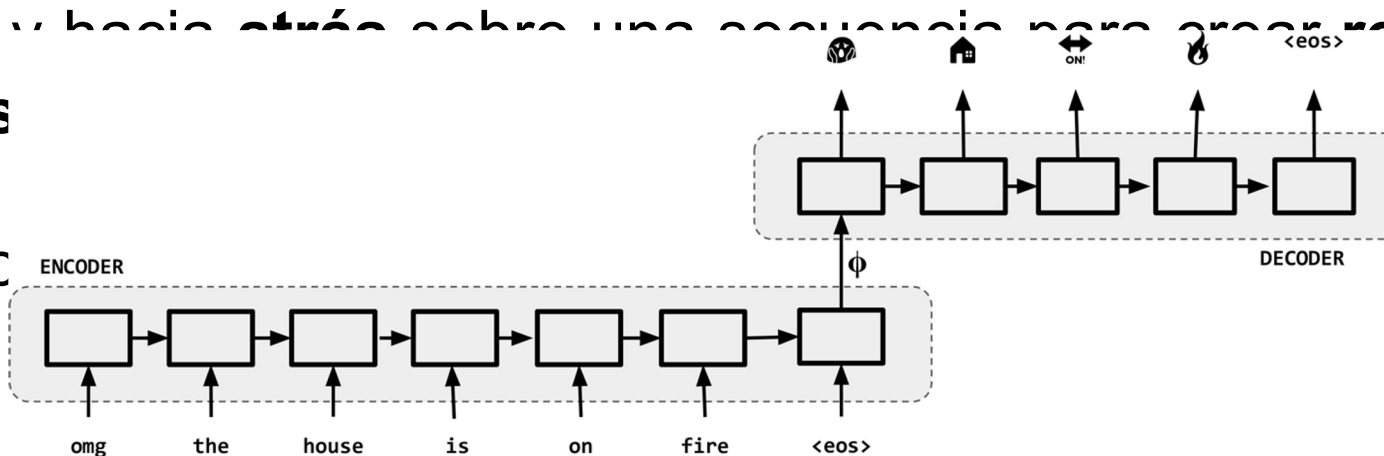


Sequence to Sequence Models (III)

- **Machine translation:** traducción de emojis como problema de predicción de S2S.
 - Podemos usar **cualquier** modelo **secuencial** (RNN, GRU o LSTM) para construir el **encoder**.

- Se suelen usar modelos **bi-direccionales** que combinan **pasadas** hacia **adelante** y **hacia atrás** sobre una secuencia para crear **representaciones más ricas**

- Por último





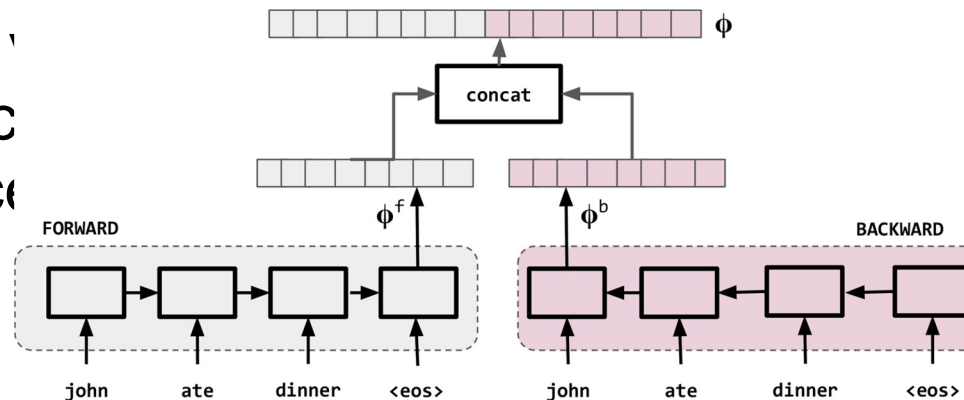
Bidirectional Recurrent Models

- Tienen la capacidad de **procesar información** tanto del **pasado** como del **futuro** de una secuencia para generar una **representación** más **completa** del **significado** de cada elemento dentro de ella.
- Ejemplo:
 - “A pesar de la demora, el servicio de atención al cliente fue excepcional.”
 - Leyendo de izquierda a derecha: podría interpretarse que hay una crítica negativa debido a la palabra "demora".
 - Los humanos **actualizamos retroactivamente** el significado de las palabras una vez que procesamos toda la frase.
 - Imitan esta capacidad humana al procesar la información tanto del contexto precedente como del siguiente.



Bidirectional Recurrent Models

- Tienen la capacidad de **procesar información** tanto del **pasado** como del **futuro** de una secuencia para generar una **representación** más **completa** del **significado** de cada elemento dentro de ella.
- Ejemplo:
 - “A pesar de la demora, el servicio de atención al cliente fue excepcional.”
 - Leyendo de izquierda a derecha: podría interpretarse que hay una crítica negativa debido a la palabra "demora".
 - Los humanos **actualizamos retroactivamente** el significado de las palabras una a una.
 - Imitan esta c contexto prec



a información tanto del



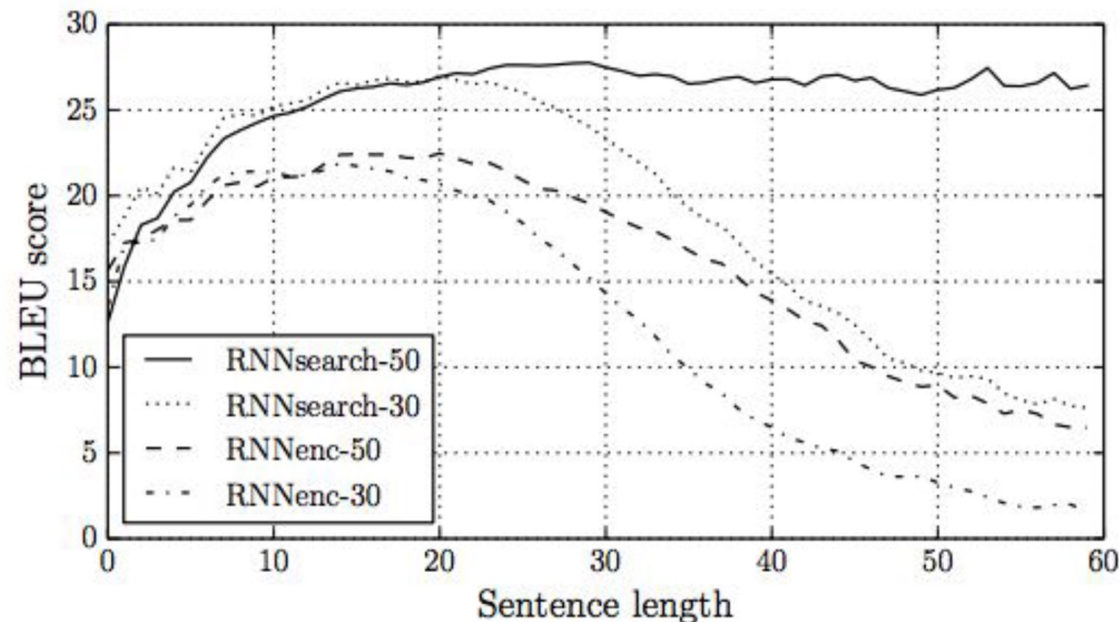
Mecanismos de Atención

- La **atención** en el procesamiento del lenguaje natural (NLP) **refleja** nuestra **capacidad humana** de **centrarnos** en **partes específicas** de la **información** mientras traducimos o procesamos el lenguaje.
- **No traducimos oraciones enteras** de una vez, sino que **relacionamos** elementos concretos de la entrada (como **palabras**) con su **significado** o correspondiente en otro **idioma**.
- Este **enfoque** selectivo, o **atención**, nos permite ser **eficientes** a pesar de nuestras **limitaciones** de **memoria**, enfocándonos en lo más **relevante** en cada momento de la comunicación.



Mecanismos de Atención

- De una forma análoga, los **modelos** de **generación** de **secuencias** deberían dar diferente **relevancia** a los **distintas partes** del **input**, y **no solo** usar el resumen o representación final de todo el input (el **último hidden state** de la red recurrente) → **Attention Mechanism**.
- Los **primeros** modelos de **machine translation**.



Los modelos terminados en search tenían atención. 50 y 30 es la longitud máxima de las traducciones de entrenamiento. [Source](#)



Mecanismos de Atención

- Basado en:

Attention mechanisms

CS 685, Fall 2022

Advanced Natural Language Processing

Mohit Iyyer

College of Information and Computer Sciences

University of Massachusetts Amherst

[Source](#)



Mecanismos de Atención

A RNN Language Model

output distribution

$$\hat{y} = \text{softmax}(W_2 h^{(t)} + b_2)$$

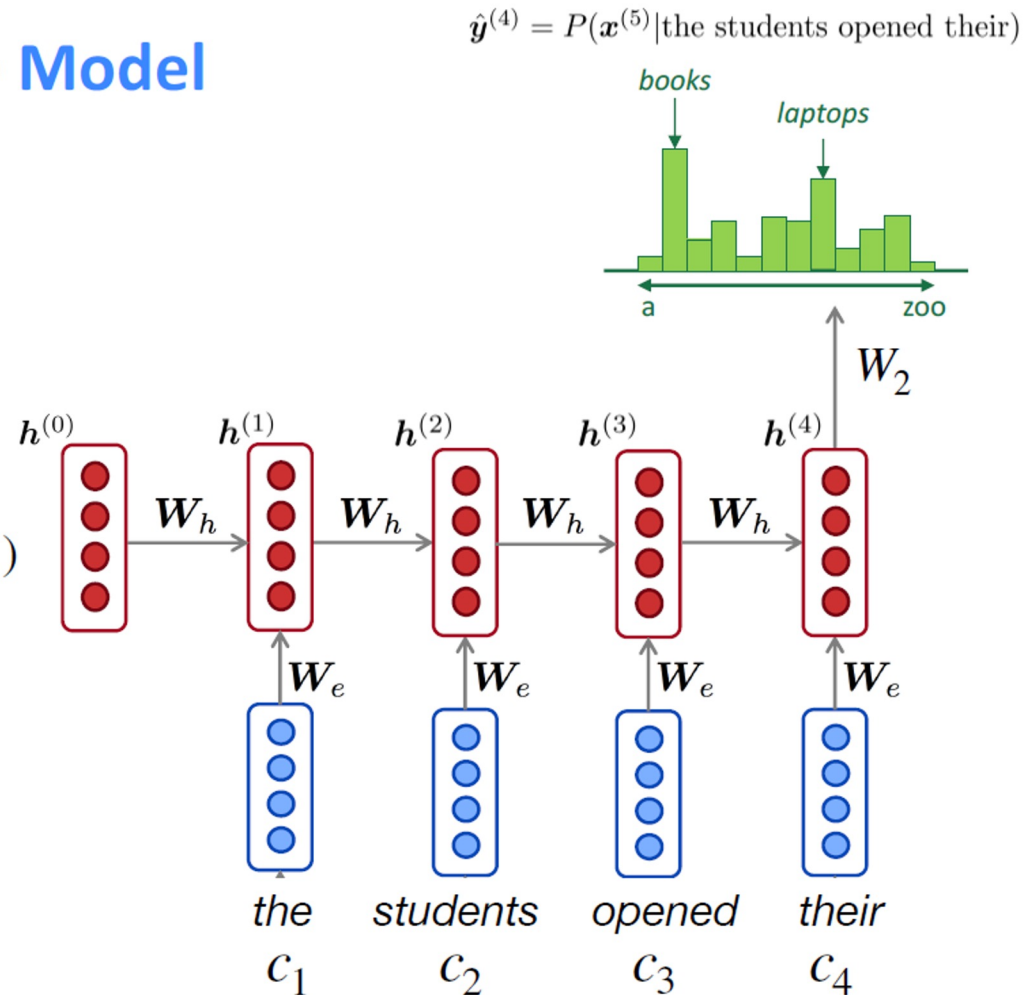
hidden states

$$h^{(t)} = f(W_h h^{(t-1)} + W_e c_t + b_1)$$

$h^{(0)}$ is initial hidden state!

word embeddings

c_1, c_2, c_3, c_4

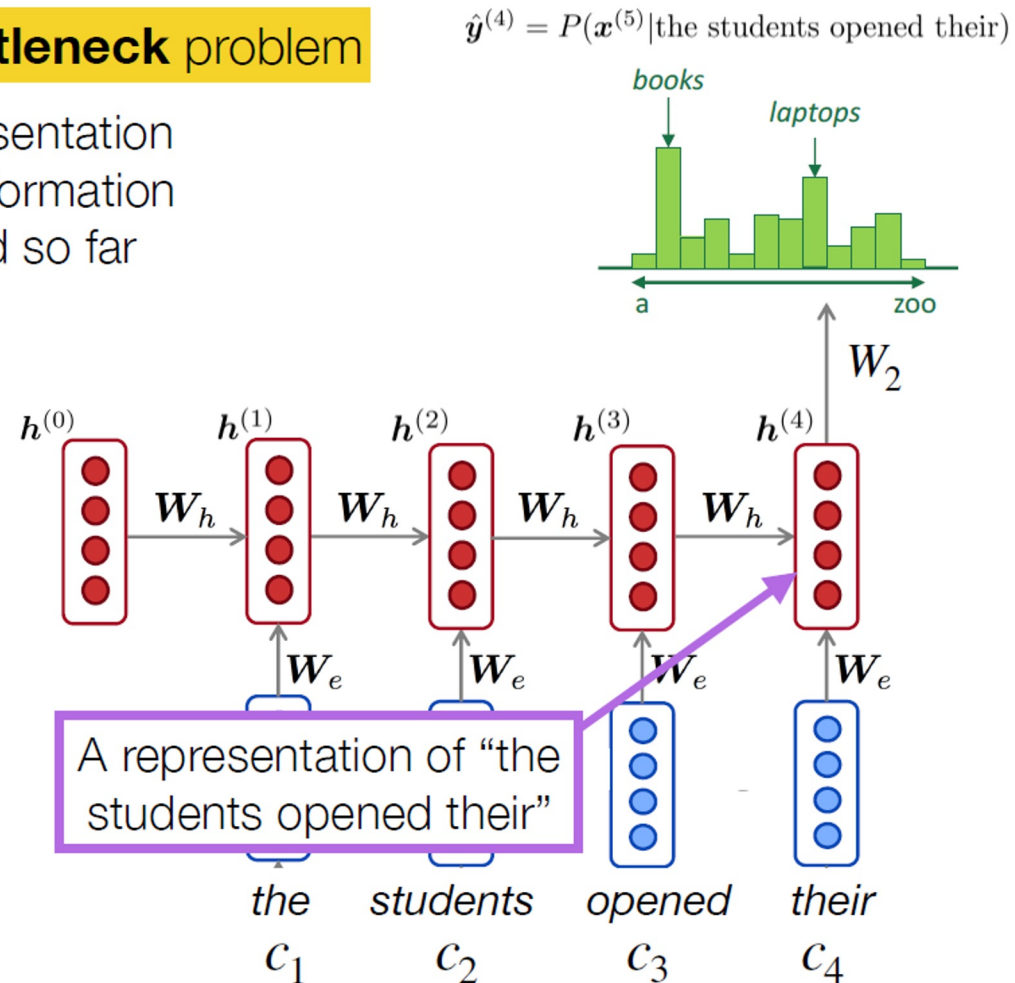




Mecanismos de Atención

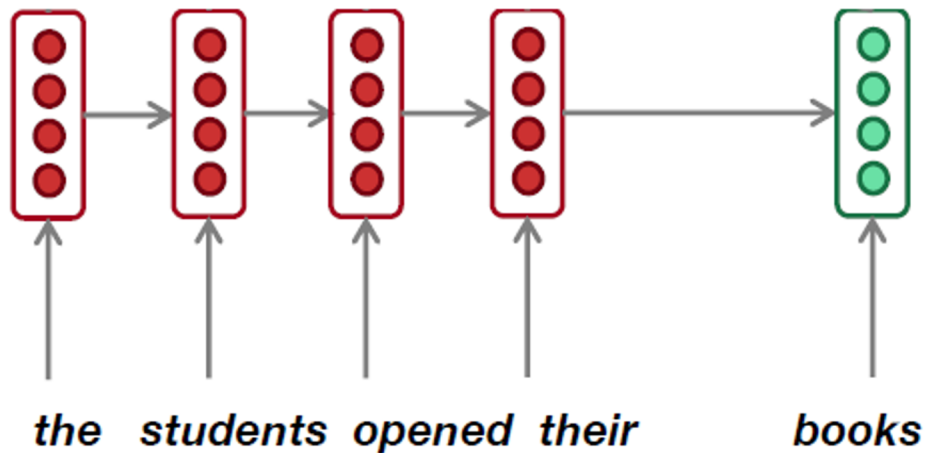
RNNs suffer from a **bottleneck** problem

The current hidden representation must encode all of the information about the text observed so far





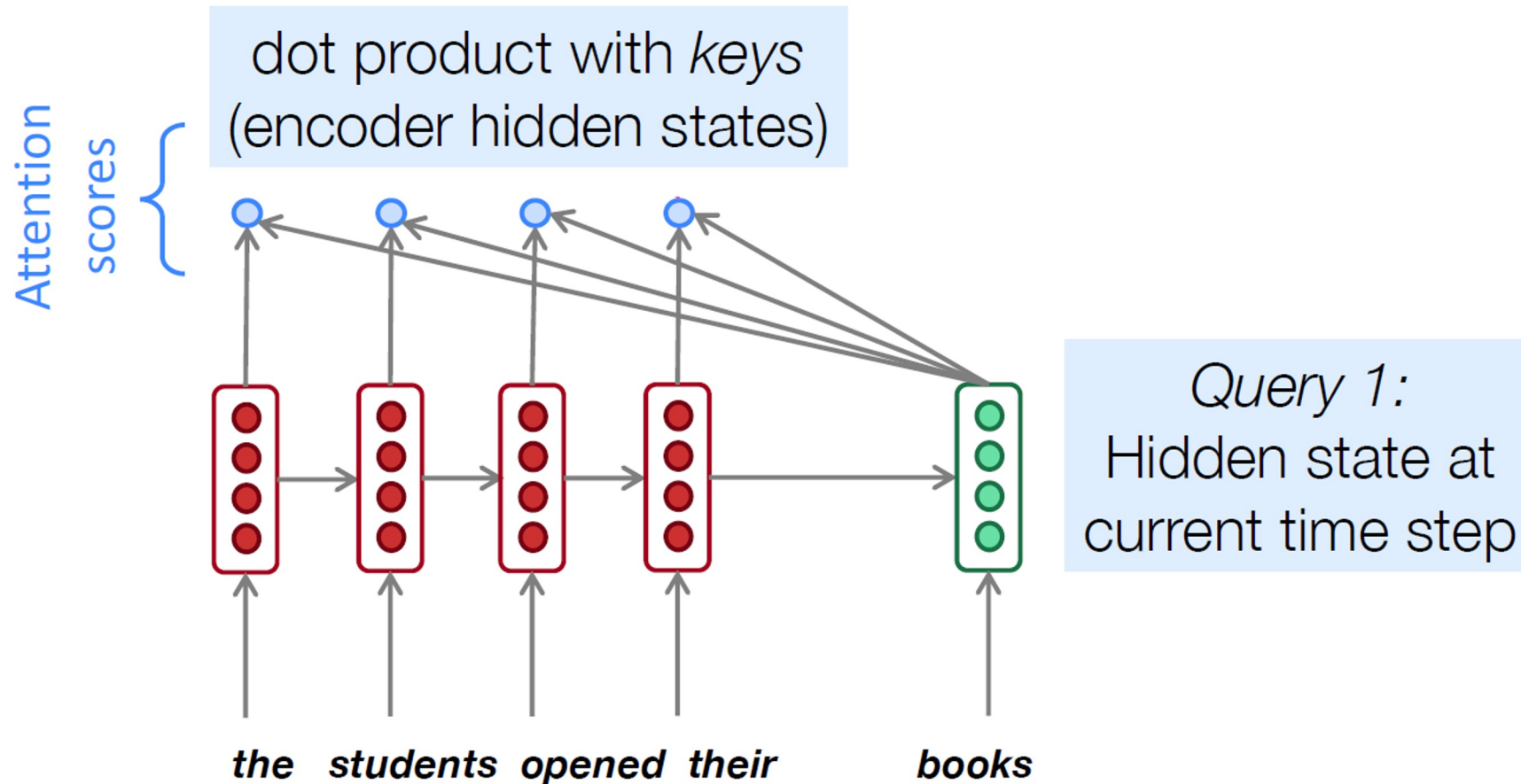
Mecanismos de Atención



Query 1:
Hidden state at
current time step

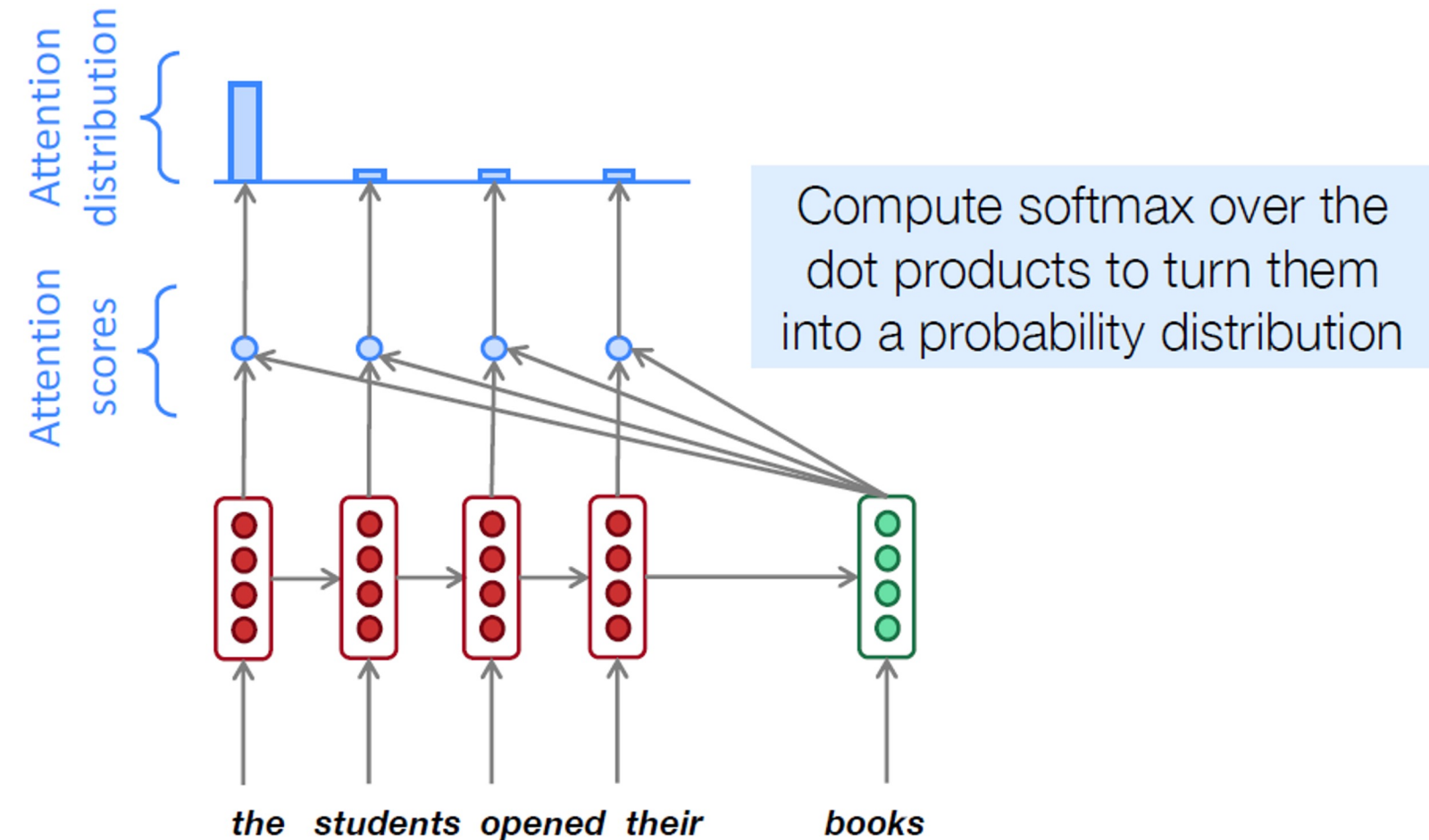


Mecanismos de Atención



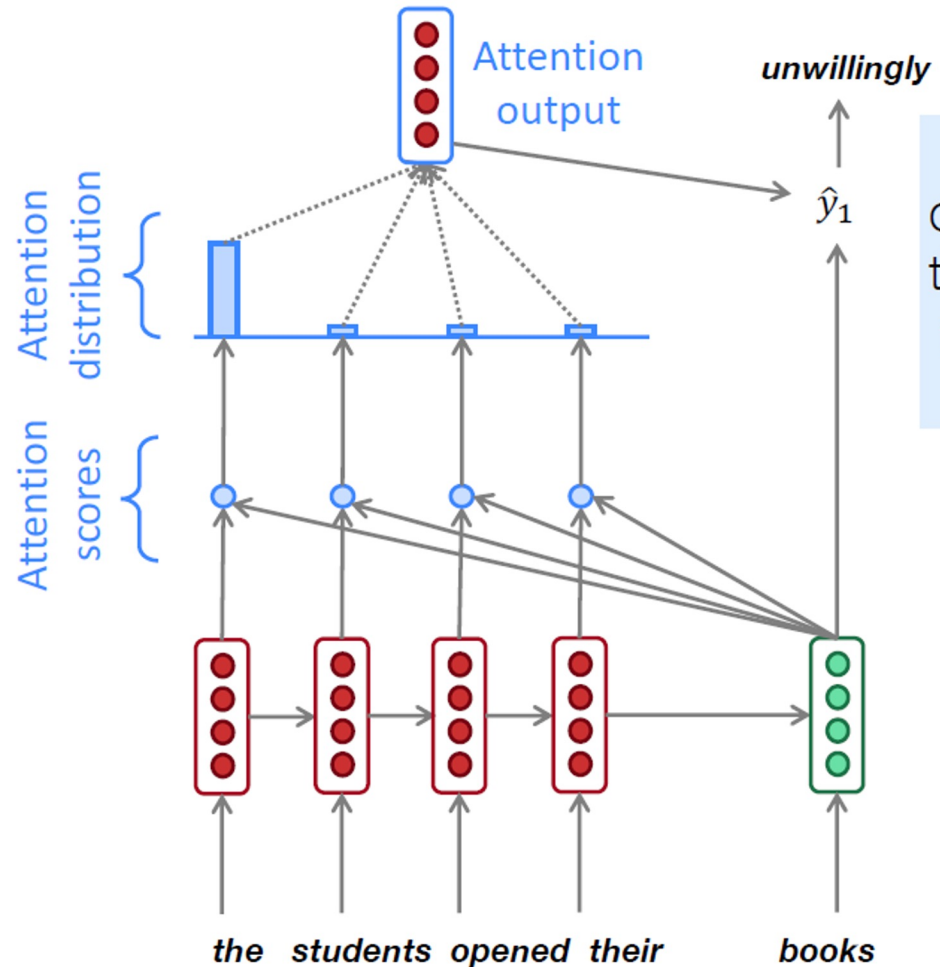


Mecanismos de Atención





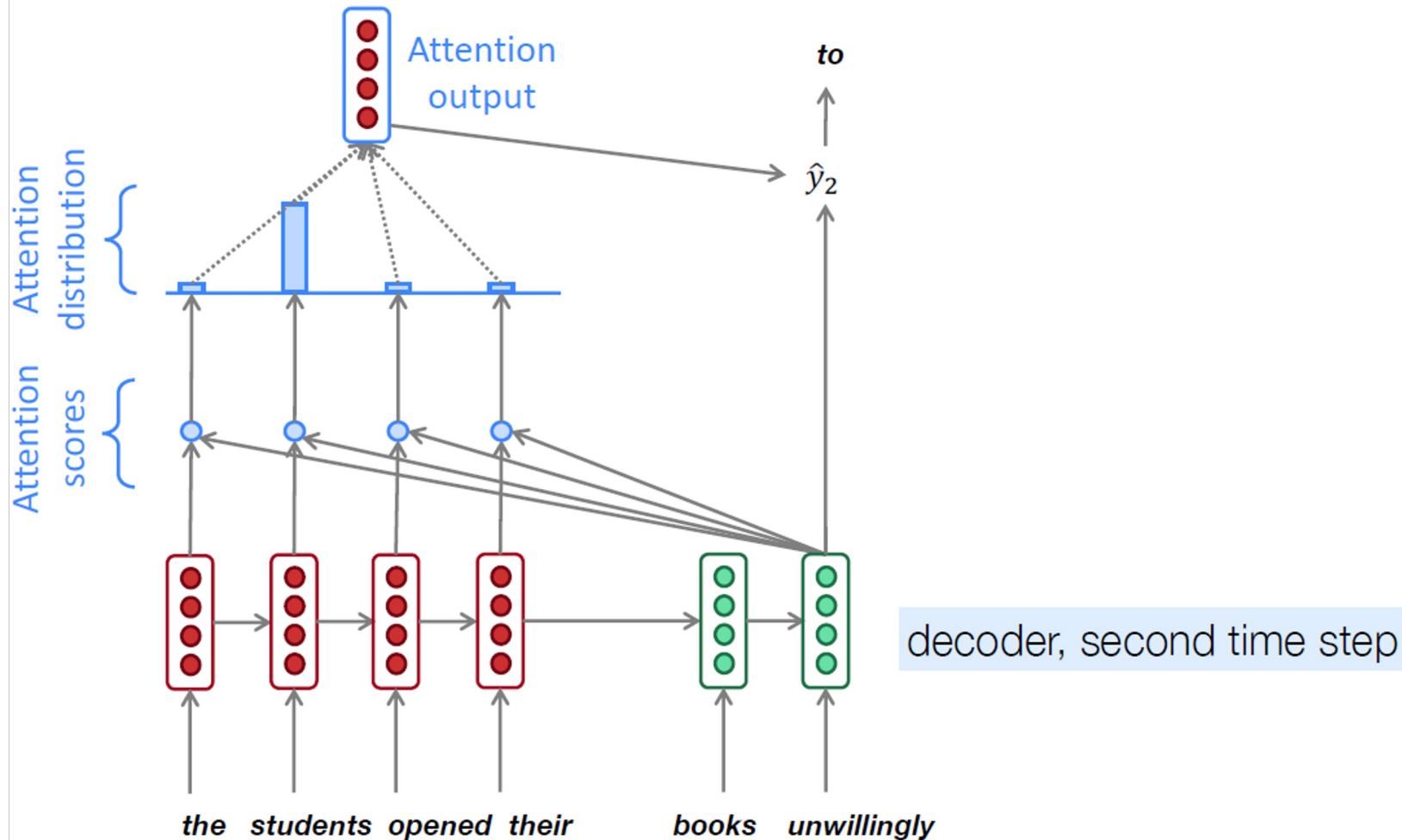
Mecanismos de Atención



Concatenate (or otherwise compose) the attention output with the current hidden state, then pass through a softmax layer to predict the next word



Mecanismos de Atención





Mecanismos de Atención

Cálculo de Atención en RNNs:

- Para cada *hidden-state*, calculamos una **puntuación de similitud** con el **estado actual** (por ejemplo, usando el *dot product*).
- Aplicamos la función **softmax** a todas las puntuaciones para obtener los **attention weights**.
- Multiplicamos cada *hidden-state* por su *attention weight* y sumamos.
- Concatenamos/Combinamos el vector de contexto con el estado actual.



Machine Translation - Dataset

- Dataset de [Tatoeba Project](#) donde tienen una gran colección de palabras traducidas:
 - [Traducciones de oraciones Inglés-Francés.](#)

Go.	Va !
Run!	Cours !
Run!	Courez !
Wow!	Ça alors !
Fire!	Au feu !
Help!	À l'aide !
Jump.	Saute.
Stop!	Ça suffit !
Stop!	Stop !
Stop!	Arrête-toi !
Wait!	Attends !
Wait!	Attendez !
I see.	Je comprends.
I try.	J'essaye.
I won!	J'ai gagné !
I won!	Je l'ai emporté !
Oh no!	Oh non !
Attack!	Attaque !
Attack!	Attaquez !
Cheers!	Santé !
Cheers!	À votre santé !
Cheers!	Merci !
Get up.	Lève-toi.



Machine Translation - Preprocesado

- Procesamiento del dataset:
 - Cada idioma de forma independiente.
 - Tokenizers específicos de idioma.
 - LowerCase()
 - Train (70%), Eval(15%), Test(15%)

```
("i", "am"), ("i", "'m"),  
("he", "is"), ("he", "'s"),  
("she", "is"), ("she", "'s"),  
("you", "are"), ("you", "'re"),  
("we", "are"), ("we", "'re"),  
("they", "are"), ("they", "'re")
```

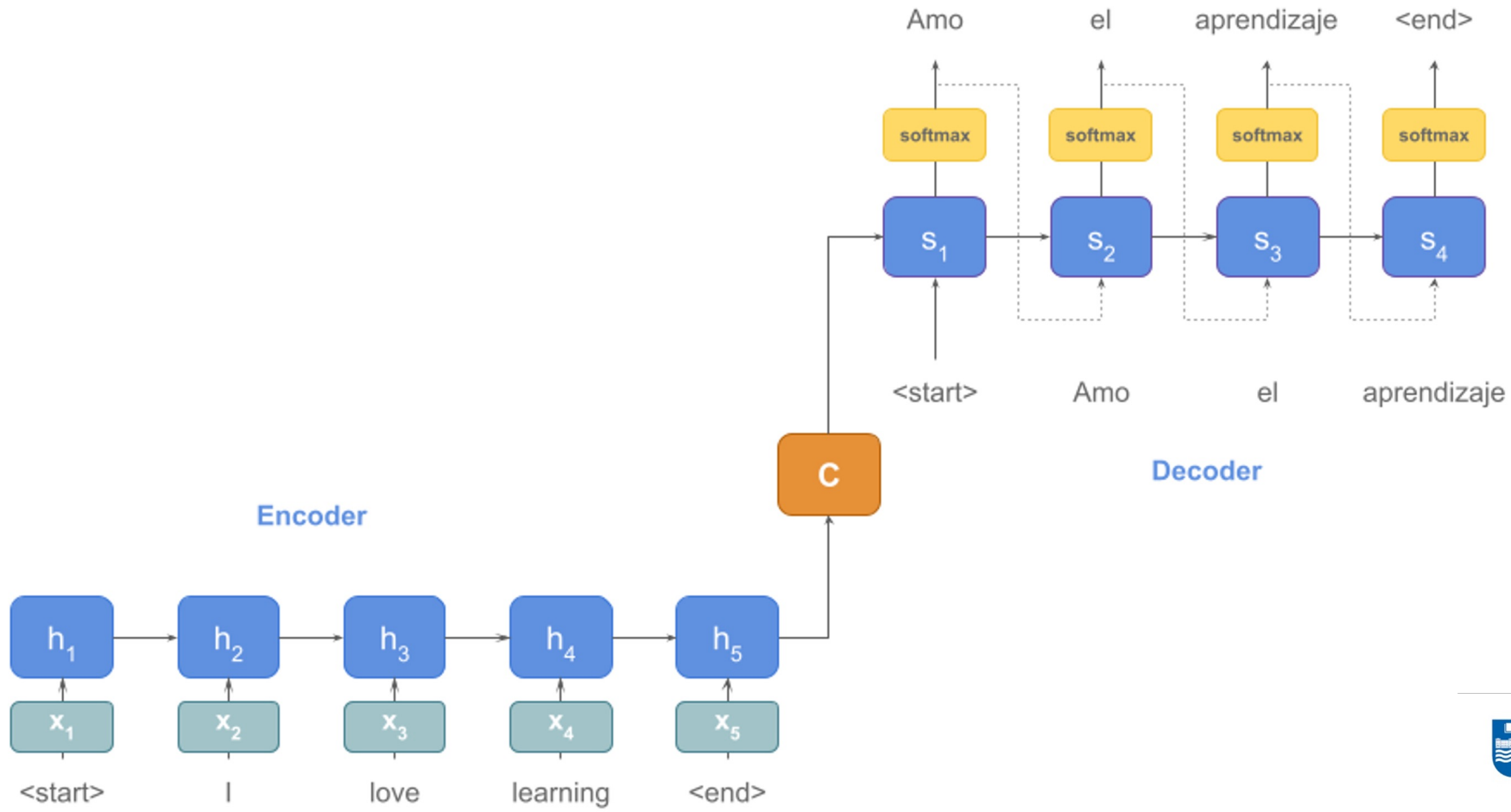


Machine Translation - Vectorizer

- Cambios en el Vectorizer:
 - Cada idioma es vectorizado de una forma diferente.
 - Idioma de origen $\rightarrow$ idioma de destino.
 - ¿Cuál es la X de entrenamiento? ¿La Y?
 - ¿Cada idioma tiene una longitud máxima diferente?
 - Cada idioma tiene un vocabulario diferente.



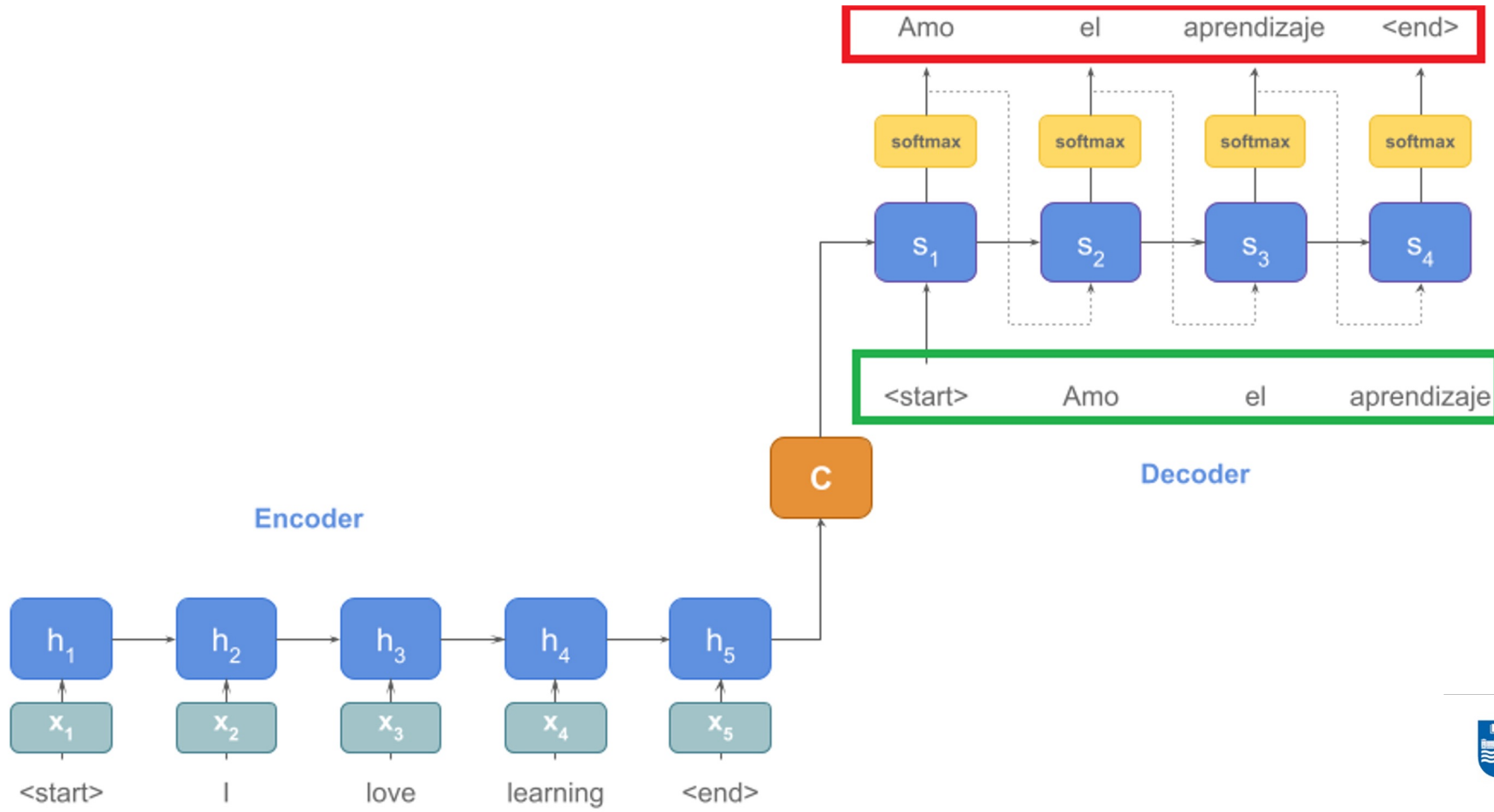
Machine Translation - Vectorizer (II)



[Source](#)



Machine Translation - Vectorizer (II)



[Source](#)



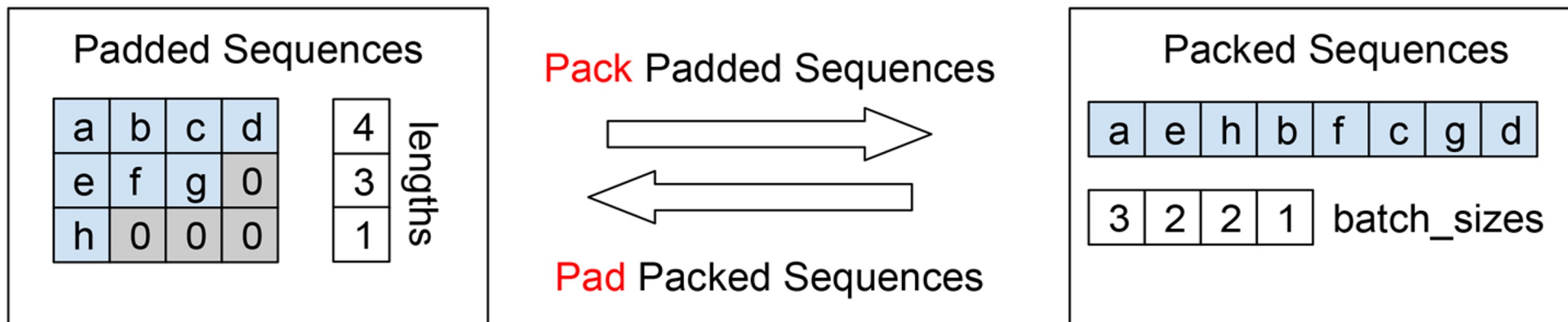
Machine Translation - Encoder

- **Encoder:** Oración en idioma de origen → Representación intermedia.
 - **Bidirectional GRU.**
 - Obtenemos dos representaciones de cada una de las palabras que son concatenadas.
 - **Packed sequences.**
 - Una forma diferente de gestionar el *padeo*.
 - **Mecanismo de atención.**
 - Cada uno de los estados del encoder sean visibles para el decoder → la importancia de cada uno de estos estados será diferente en cada uno de los time steps/estados/palabras del decoder.



Machine Translation - Packed Sequences

- Mucho más eficiente computacionalmente.
 - Para esto necesitamos también como input la longitud de las oraciones que queremos traducir.
 - [Explicación más detallada aquí.](#)



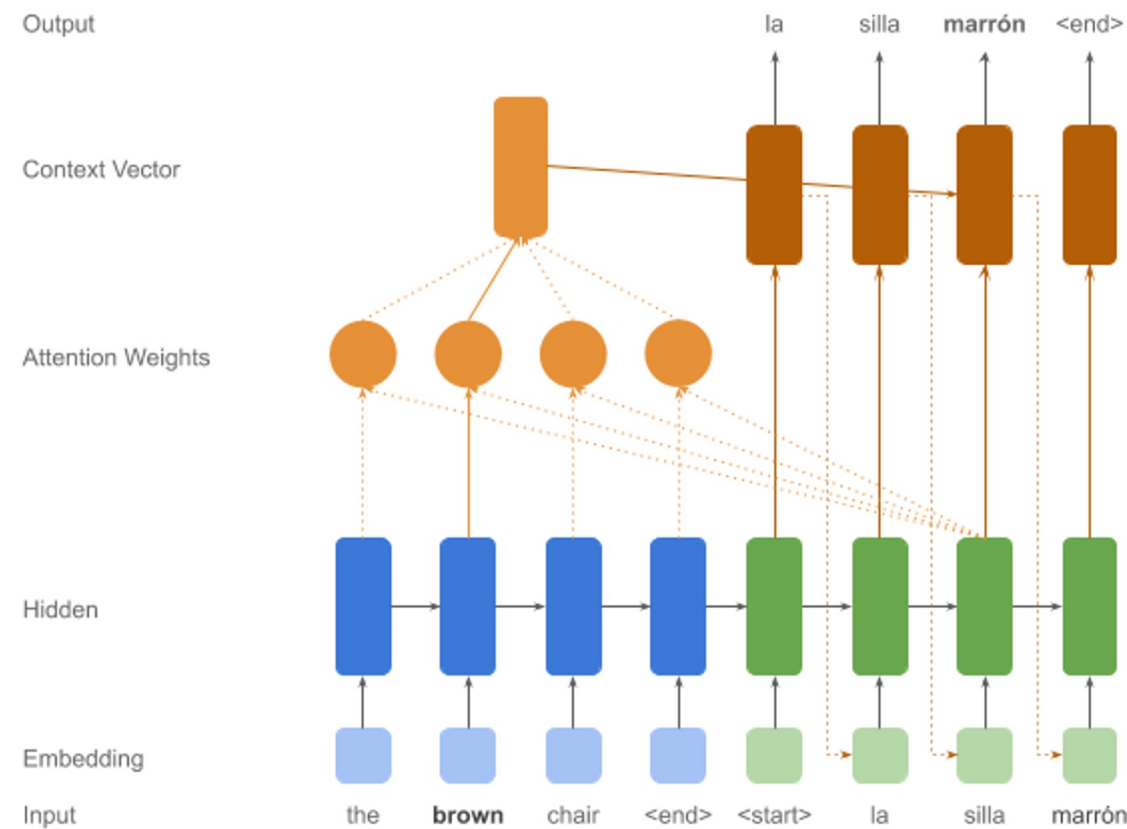


Machine Translation - Decoder

- **Decoder:** Representación intermedia + atención → Oración en idioma objetivo.
 - **Initial state** → **Representación intermedia de la oración generada por el encoder.**
 - **Tamaño:** [64,128]
 - **GRU**
 - Hacemos uso de GRUCell en lugar de GRU para implementar la atención en cada estado.
 - **Mecanismo de atención.**
 - Hacemos uso de la atención en cada time step, calculando diferentes context vectors en cada estado.
 - El tamaño del tensor que representa el encoder en cada uno de los estados es **[64, X, 128]** donde X es la longitud variable



Machine Translation - Complete



[Source](#)



Bibliografía

- Hapke, H., Howard, C., & Lane, H. (2019). Natural Language Processing in Action: Understanding, analyzing, and generating text with Python. Simon and Schuster.
- Peter Bloem, David Romero , VU University Amsterdam, <https://dlvu.github.io/>
- Rao, D., & McMahan, B. (2019). Natural language processing with PyTorch: build intelligent language applications using deep learning. " O'Reilly Media, Inc.".
- Julia Hockenmaier, CS447, University of Illinois, <https://courses.grainger.illinois.edu/CS447/sp2023/Slides/Lecture05.pdf>
- CS 230 - Deep learning, Stanford University, <https://stanford.edu/~shervine/teaching/cs-230/cheatsheet-recurrent-neural-networks>
- Recurrent Neural Networks Report, Mahmud Oyinloye, 2022: <https://medium.com/@le.oasis/recurrent-neural-networks-report-c70e6f05cc9e>
- Attention mechanisms, Mohit Iyyer, College of Information and Computer Sciences, University of Massachusetts Amherst: https://people.cs.umass.edu/~miyyer/cs685_f22/slides/04-attention.pdf